



TRABAJO FIN DE GRADO
GRADO EN INGENIERIA INFORMATICA

Sistema de gestión automática de plazas de parking

Autor

Javier García Pérez

Director

Miguel Damas Hermoso



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Junio de 2025

Sistema de gestión automática de plazas de parking

Javier Garcia Perez

Palabras clave: Python, Flet, MongoDB, API, Flask, Detector Matriculas, Parking, UI, software libre

Resumen

Este Trabajo Fin de Grado (TFG) tiene como objetivo el diseño y desarrollo de un sistema destinado a la gestión automática del parking de la Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación (ETSIIT), encargado de recopilar y gestionar los datos de los vehículos que acceden al mismo. El sistema busca facilitar el acceso al parking, evitar problemas en la entrada/salida y contribuir a la informatización del edificio.

Para su desarrollo, se ha utilizado Python junto con el framework Flet, que permite crear una interfaz atractiva y funcional. Además, se emplea MongoDB para gestionar todos los datos recopilados por la aplicación. Se ha priorizado el uso de herramientas de código abierto y tecnologías accesibles con el fin de mantener el coste del sistema lo más bajo posible. Aunque esta aplicación ha sido diseñada específicamente para el parking de la ETSIIT, es fácilmente adaptable a cualquier otro aparcamiento con una estructura similar, ofreciendo un sistema fiable y eficiente para el seguimiento de los vehículos que lo utilizan.

Automatic parking space management system

Javier Garcia Perez

Keywords: Python, Flet, MongoDB, API, Flask, Detector Matriculas, Parking, UI, software libre

Abstract

This Final Degree Project (TFG) focuses on the design and development of a system for the automatic management of the parking lot at the School of Computer and Telecommunication Engineering (ETSIIT). The system collects and manages data on the vehicles accessing the parking area. Its goal is to streamline access, prevent entry/exit-related issues, and contribute to the digitalization of the school facilities.

The system has been developed using Python along with the Flet framework, which enables the creation of a clean, and functional user interface. MongoDB is used to manage all the data collected by the application. Open-source tools and accessible technologies have been prioritized to keep the overall cost of the system as low as possible. Although the application is specifically designed for the ETSIIT parking lot, it can be easily adapted to any parking facility with a similar structure, providing a reliable and efficient vehicle tracking system.

Yo, **Javier García Pérez**, alumno de la titulación GRADO EN INGENIERÍA INFORMATICA de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 76067757H, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Javier García Pérez

Granada, a 6 de junio de 2025

D. **Miguel Damas Hermoso**, Profesor del Departamento de Ingeniería de Computadores, Automática y Robótica

Informo:

Que el presente trabajo, titulado ***Sistema de gestión automática de plazas de parking***, ha sido realizado bajo mi supervisión por **Javier García Pérez**, y autorizo la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a Junio de 2025.

El tutor:

Agradecimientos

Quiero agradecer sobre todo a mis padres, por estar siempre ahí cuando lo he necesitado y cuando no, por su implicación para que me vayan bien las cosas, siempre están ahí para ayudarme en todo lo que pueden.

A toda mi familia por apoyarme siempre para sacar la carrera y para todo.

A mis amigos más cercanos que nos hemos ayudado a sacar todo adelante.

Índice

1. Introducción	15
1.1. Contexto	15
1.2. Motivación	15
1.3. Situación actual y futura	16
1.4. Objetivos	19
1.5. Estructura de la memoria	19
2. Estado del arte	20
2.1. Aplicaciones móviles	20
2.2. Plataformas integrales de gestión	21
2.3. Comparativa de las soluciones existentes	21
2.4. Líneas de mejora identificadas	22
2.5. Posicionamiento del proyecto	22
3. Planificación y Presupuesto	23
3.1. Planificación	23
3.2. Costes	25
3.2.1. Coste de personal	25
3.2.2. Coste de material	25
3.2.3. Coste del despliegue	26
3.2.4. Costes indirectos	27
3.2.5. Resumen de costes	27
4. Análisis de requisitos	28
4.1. Requisitos funcionales	28
4.2. Requisitos no funcionales	28
4.3. Requisitos de información	29
5. Diseño de la aplicación	30
5.1. Herramientas y tecnologías	30
5.2. Justificación de la elección tecnológica	31
5.3. Diseño de la base de datos	33
5.3.1. Colección vehicles	34
5.3.2. Colección logs	34
5.3.3. Colección users	35
5.4. Interfaz del Usuario	36
5.4.1. Vista principal	36
5.4.2. Vista del Panel del Administrador	37
5.4.3. Vistas de Datos	39
5.4.4. Gráficas y Estadísticas	41

5.4.5. Navegación y Accesibilidad	42
5.4.6. Autenticación de Usuarios	43
6. Implementación	46
6.1. Estructura de archivos y preparación del entorno	46
6.1.1. Preparación del entorno	47
6.2. Arquitectura general del sistema	48
6.2.1. Diagrama de Arquitectura	49
6.2.2. Flujo de Datos Principal	50
6.3. Backend	51
6.3.1. Inicialización del Sistema	52
6.3.2. API	53
6.3.3. Modelos	58
6.3.4. Servicios	59
6.4. Detección de Matrículas	69
6.4.1. Integración de Modelos de Visión por Computador	69
6.4.2. Flujo de Detección	70
6.4.3. Ejemplo de Detección	71
6.5. Base de Datos	72
6.5.1. Estructura General	72
6.5.2. Inserción de Datos	73
6.5.3. Consulta de Datos	74
6.6. Frontend	75
6.6.1. Inicialización del Sistema	75
6.6.2. Vistas	76
6.7. Componentes Frontend	85
6.7.1. Gráficas	85
6.7.2. Barra de Navegación	89
6.8. Autenticación de Usuarios	91
7. Pruebas y Validación	93
7.1. Contenido Responsive	93
7.2. Grabaciones del Resultado Final	98
8. Conclusiones	99
8.1. Objetivos Alcanzados	99
8.2. Vías Futuras	99
8.3. Valoración de Flet	101
8.4. Valoración Personal	101

1 | Introducción

1.1. Contexto

En los próximos años, la Escuela ETSIT será remodelada. Se construirá un nuevo edificio donde están ahora mismo las aulas prefabricadas y se unirá el parking actual con el nuevo parking que se va a construir, el parking se quedaría con un diseño unidireccional.

Los vehículos entran por la entrada actual, recorren la *Zona actual* (Zona 1), giran 90 ° hacia la *Zona nueva* (Zona 2) y salen por un único punto. Este diseño tiene un problema importante, impide retroceder ni dar media vuelta en caso de encontrar la *Zona 2* completa.

1.2. Motivación

Esta situación refleja un reto común en muchos parkings con diseños similares, la falta de información en tiempo real sobre la disponibilidad de plazas. Imagina a un profesor llegando tarde a una reunión importante, entra al parking con la intención de aparcar en la *Zona 2* (la más cercana a los despachos de profesores y áreas administrativas), pero al descubrir que esta zona está llena, tiene que dar una vuelta innecesaria, perder tiempo y aumentar el estrés.

Esto no solo afecta su día, sino que también aumenta el riesgo de accidentes por maniobras improvisadas dentro del parking o fuera. Además, en horas punta, la acumulación de vehículos intentando entrar y salir podría entorpecer la movilidad general cercana a la escuela o incluso bloquear el acceso a servicios de emergencia.

Este proyecto surge como respuesta a ese problema. Se propone el desarrollo de un sistema inteligente que detecte automáticamente el vehículo y la matrícula al entrar, al pasar por las distintas zonas del parking y al salir, y proporcione en tiempo real información sobre la ocupación de cada zona del parking, ya sea mediante una pantalla informativa en el propio parking o una aplicación móvil. Así, si la *Zona 2* está completa, el conductor lo sabrá incluso antes de entrar al parking, evitando pérdidas de tiempo y optimizando el flujo de vehículos.

Además, el sistema registrará los datos de los vehículos en una base de datos, lo que permitirá al personal de la universidad analizar patrones de uso, zonas más demandadas, identificar horarios de más tráfico, detectar vehículos no autorizados y planificar posibles mejoras o ampliaciones del aparcamiento.

Más allá de su parte tecnológica, este proyecto tiene un objetivo mayor, contribuir a un entorno universitario más funcional, tecnológico, seguro y adaptado a las necesidades de profesores, personal administrativo, estudiantes y visitantes.

Esta idea nace de conversaciones con el personal de mantenimiento de la Escuela, quienes me avisaron de los problemas del futuro diseño del parking y la necesidad de soluciones que mitiguen sus desventajas.

1.3. Situación actual y futura

Para comprender mejor el problema al que se enfrenta la Escuela, es útil ver el diseño actual y el previsto del aparcamiento. En la Figura 1.1 se muestran los planos del sótano del nuevo edificio, mientras que en la Figura 1.2 son los planos del edificio actual. La zona blanca en la parte superior del plano del nuevo edificio es donde se van a unir ambos edificios, donde se completa el futuro sistema del parking unidireccional.



Figura 1.1: Plano del Sótano del nuevo edificio.

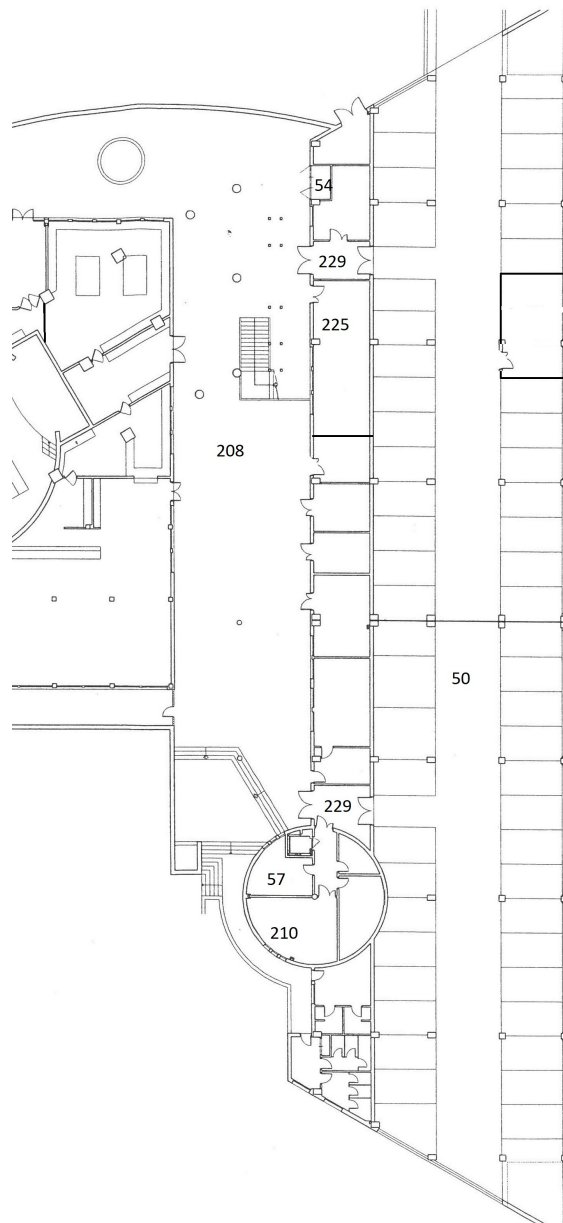


Figura 1.2: Plano del Sótano del edificio actual.

Como se puede ver en los planos del nuevo edificio, los nuevos despachos (cuadrados azules en la sección amarilla) estarán pegados a la nueva zona del parking. Además, se habilitará un nuevo recorrido (más directo y corto) para acceder desde el parking a los despachos actuales. Este nuevo camino comienza en la puerta situada junto a la rampa de salida de vehículos de la nueva zona, se recorre el pasillo y hay que subir por las escaleras o el ascensor hasta la segunda

planta. Allí se construirá una pasarela que conectará con la segunda planta del edificio de los despachos actuales, como se muestra en la siguiente Figura 1.3. Esto hará que seguramente todos los profesores/personal administrativo quieran aparcar en la nueva zona para llegar antes a sus despachos.

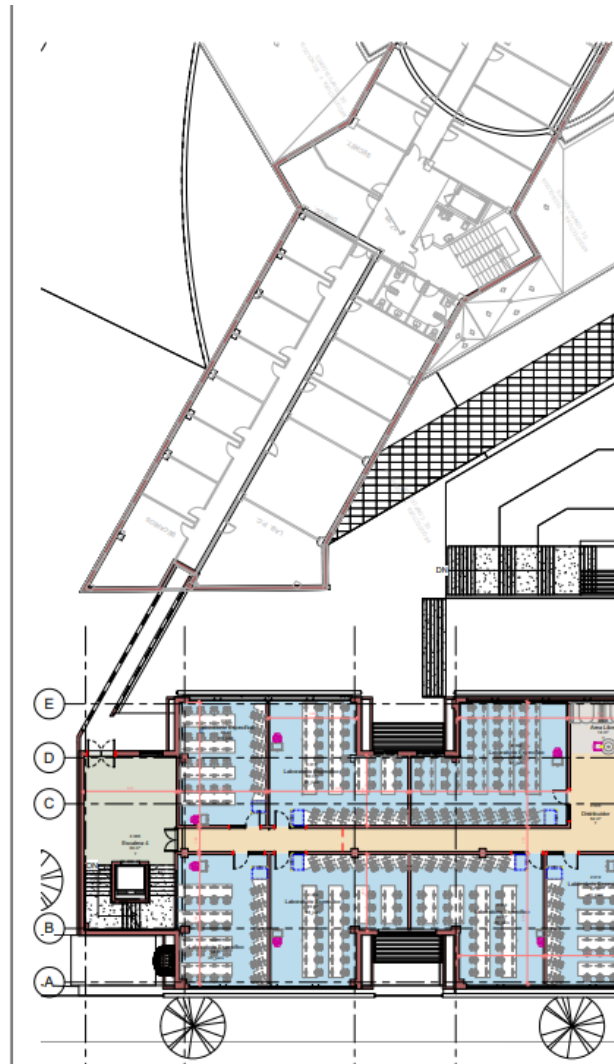


Figura 1.3: Plano de la pasarela de la 2ª planta del nuevo edificio.

Este nuevo diseño introduce un problema importante como muchos profesores van a querer aparcar lo mas cerca posible, provocará que si un vehículo entra al aparcamiento y la Zona 2 está completa, no tendrá forma de retroceder ni cambiar de dirección. Esto genera pérdidas de tiempo innecesarias, situaciones de estrés y un mayor riesgo de accidentes por maniobras improvisadas dentro y fuera del parking.

1.4. Objetivos

Estos son los objetivos que me he marcado para el desarrollar el TFG:

Objetivo general

Desarrollar un sistema multiplataforma, basado en visión artificial, capaz de supervisar automáticamente la disponibilidad de plazas en un parking unidireccional y mostrar los datos recopilados.

Objetivos específicos

1. Detectar matrículas en tiempo real con una precisión mínima del 95 %.
2. Mostrar en tiempo real la ocupación de *Zona 1* y *Zona 2*.
3. Registrar en una base de datos las entradas, salidas y datos sobre el vehículo.
4. Compatibilidad multiplataforma (Windows, Linux, móviles...etc).
5. Desarrollar una solución competitiva y económica frente a otras alternativas del mercado.

1.5. Estructura de la memoria

La memoria se organiza en ocho capítulos:

- **Capítulo 1: Introducción.** Presentación del proyecto y motivación
- **Capítulo 2: Estado del arte.** Análisis de soluciones actuales y oportunidades de mejoras.
- **Capítulo 3: Planificación y presupuesto.** Organización temporal del desarrollo y estimación de costes.
- **Capítulo 4: Análisis de requisitos.** Especificación de requisitos funcionales, no funcionales y de información.
- **Capítulo 5: Diseño.** Elección de tecnologías usadas con su justificación y el diseño del sistema.
- **Capítulo 6: Implementación.** Desarrollo práctico del sistema: backend, frontend y base de datos.
- **Capítulo 7: Pruebas y validación.** Test de diseño y comportamiento del sistema.
- **Capítulo 8: Conclusiones** Resumen de objetivos, trabajos futuros y valoraciones.

2 | Estado del arte

La identificación de vehículos es un requisito legal en España (Ley 40/2002, de 14 de noviembre), especialmente en parkings donde se cede un espacio como actividad mercantil con deberes de vigilancia y custodia a cambio de un precio por tiempo de estacionamiento [4]. A continuación, se analizan las empresas más representativas, clasificadas según su enfoque principal: aplicaciones de usuario o plataformas integrales de gestión.

2.1. Aplicaciones móviles

Estas aplicaciones se centran en la experiencia del conductor, ofreciendo búsqueda de plazas, pago automático en parkings y registro de matrícula desde el móvil:

- **EasyPark – CameraPark**

Permite pre-registrar la matrícula y pagar el estacionamiento desde el teléfono. Las cámaras toman la matrícula al entrar y al salir, y el cobro se realiza automáticamente desde la cuenta vinculada. EasyPark no instala sus propias cámaras ni desarrolla algoritmos específicos para el reconocimiento de matrículas, utiliza directamente la infraestructura ya instalada por los operadores de los aparcamientos. Esta información fue confirmada por Jaime Requeijo, director general de EasyPark España, en una entrevista donde destacó que la empresa utiliza los sistemas de reconocimiento de matrícula ya existentes en los parkings para ofrecer su servicio CameraPark. [12]

- **TelPark**

Además de ofrecer pago remoto en zonas reguladas con la funcionalidad *Entrada Express*, Telpark permite reservas en tiempo real y puntos de recarga para vehículos eléctricos. Telpark opera en aparcamientos gestionados por Empark, una de las mayores empresas de gestión de parkings en Europa. Empark proporciona y mantiene la infraestructura necesaria para el control de accesos, incluyendo las cámaras y el sistema de reconocimiento de matrículas.

La funcionalidad *Entrada Express* de Telpark permite a los usuarios acceder y salir de los aparcamientos sin necesidad de recoger un ticket ni realizar pagos en cajeros. Al activar esta opción, el sistema reconoce automáticamente la matrícula del vehículo al entrar y salir del aparcamiento, facilitando un proceso de estacionamiento más rápido y sin contacto [13].

Ambas empresas comparten una característica fundamental: delegan la tarea de reconocimiento visual en sistemas preexistentes dentro de los aparcamientos.

tos, evitando así la necesidad de desarrollar soluciones propias de detección del vehículo.

2.2. Plataformas integrales de gestión

Estas empresas combinan hardware, software y servicios para ofrecer una solución completa al operador del parking:

- **Quercus Technologies**

Ofrece un sistema integral listo para su implementación que incluye cámaras LPR (License Plate Recognition) con una precisión superior al 99%, sensores de ocupación y una plataforma de gestión llamada BirdWatch, la cual permite generar listas blancas y controlar accesos de manera eficiente [10].

La empresa Rekor no está orientada solo a parkings, sino también a la gestión de tráfico y seguridad vial en entornos urbanos:

- **Rekor**

Rekor Scout® es un software de la empresa Rekor que se encarga del reconocimiento automático de matrículas (LPR) y vehículos a través de cualquier cámara IP, de tráfico o de seguridad. Gracias a su arquitectura basada en inteligencia artificial y aprendizaje automático, es capaz de identificar matrículas, así como datos del vehículo como marca, modelo, color y dirección de desplazamiento. Los resultados los muestran en una interfaz web accesible desde la nube o mediante implementación local. Además, ofrece funcionalidades como búsquedas avanzadas, alertas en tiempo real mediante listas de vehículos de interés y almacenamiento de datos, todo ello orientado a mejorar la seguridad pública y optimizar la gestión del tráfico. [11].

2.3. Comparativa de las soluciones existentes

Empresa	Hardware propio	App usuario final	Solución integral
EasyPark - CameraPark	No	Sí	Parcial
TelPark	No	Sí	Parcial
Quercus Technologies	Sí	No	Sí
Rekor	Sí	Sí	Sí

Tabla 2.1: Comparativa de soluciones existentes para gestión de parkings

Nota: "Solución integral" indica si la empresa ofrece tanto hardware como software propio para la gestión completa del parking. "Parcial" indica que sólo aportan parte del sistema o se integran con infraestructura preexistente.

2.4. Líneas de mejora identificadas

Del análisis anterior se sacan varias oportunidades para mejorar las soluciones actuales:

- *Multiplataforma*: muchas aplicaciones móviles no tienen versiones nativas para Windows o Linux, o requieren navegador web para su uso.
- *Información en tiempo real*: Es clave evitar perder tiempo, esto se soluciona mediante interfaces que actualicen el estado de plazas de ambas zonas al instante.
- *Software libre*: fomentar la transparencia, flexibilidad y posibilidad de adaptación mediante el uso de tecnologías y licencias abiertas.

2.5. Posicionamiento del proyecto

Este trabajo propone una aplicación multiplataforma (Windows, Linux, Android, IOS, etc.) que:

- Muestra la ocupación de plazas de las diferentes zonas del parking en tiempo real.
- Utiliza cámaras propias y software basado en tecnologías de visión artificial de código abierto, integrando herramientas como Ultralytics y Fast-ALPR.
- Es software libre, facilitando la transparencia y adaptación a diferentes entornos.
- Implementación del sistema con un coste optimizado, siendo una alternativa competitiva y flexible frente a soluciones comerciales.

Con esta propuesta se cubren las carencias de las soluciones actuales, aportando compatibilidad, rendimiento en tiempo real y transparencia gracias a licencias abiertas.

3 | Planificación y Presupuesto

En cualquier proyecto es importante tener claro el camino a seguir. La planificación ayuda a organizar los pasos que se deben dar, repartir bien los recursos y marcar unos plazos realistas. Además, hacer una buena estimación del presupuesto permite saber cuánto va a costar todo, tomar mejores decisiones y asegurarse desde el principio de que el proyecto se puede llevar a cabo sin problemas.

3.1. Planificación

En esta sección explico la planificación que usé para realizar este proyecto. Para ello, he hecho un diagrama de Gantt Figura 3.1 que permite ver de forma clara las distintas etapas del proyecto.

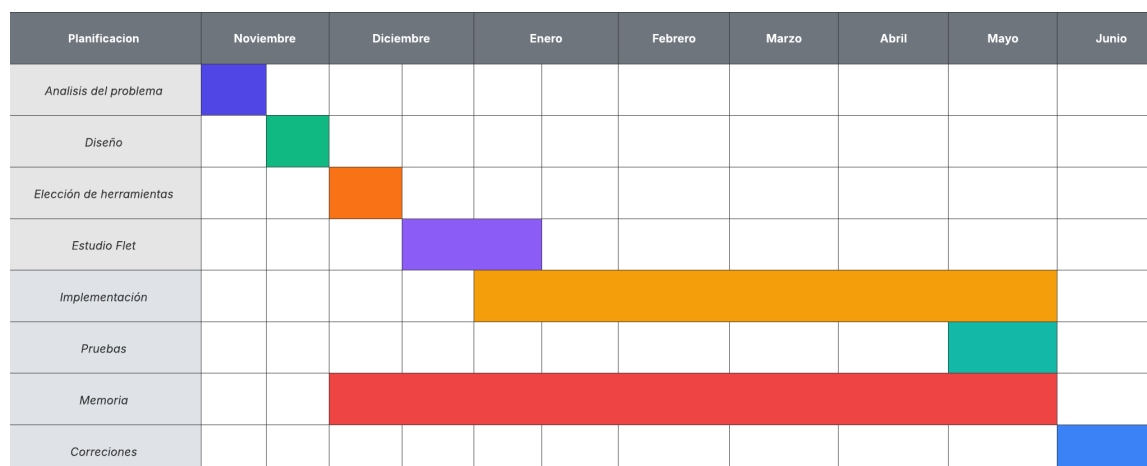


Figura 3.1: Diagrama de Gantt

El proyecto se dividió en las siguientes etapas:

1. **Análisis del problema:** En esta fase definí el problema a resolver, teniendo en cuenta las limitaciones técnicas, el contexto del parking y todas las soluciones posibles desde el punto de vista técnico y práctico.
2. **Diseño:** Una vez definido el problema, empecé con el diseño de la solución, incluyendo tanto el funcionamiento general del sistema como las vistas de la interfaz y arquitectura interna, desde que se inician las cámaras hasta que se captura la matrícula y se envía a la base de datos.
3. **Elección de herramientas:** Desde el principio tuve claro que quería implementar la parte del backend en Python por la biblioteca de Ultralytics.

Sin embargo, para el frontend tuve muchas dudas entre utilizar tecnologías que ya conocía, como React, o seguir el consejo de mi tutor y probar Flet, a pesar de no haberlo utilizado nunca. Finalmente, me decidí por Flet, ya que, al tratarse de una tecnología bastante nueva y estar orientada a crear interfaces multiplataforma, encajaba muy bien con el objetivo de desarrollar una aplicación de parking accesible desde cualquier dispositivo/pantalla.

4. **Estudio de Flet:** Al tratarse de un framework emergente, necesité dedicar tiempo a familiarizarse con su funcionamiento, componentes y capacidades. Esta etapa incluyó tanto el estudio de documentación oficial , que es muy clara y detallada, como a ver tutoriales de youtube y la realización de pequeños proyectos de prueba.
5. **Implementación:** Se desarrolló progresivamente la aplicación, integrando los distintos módulos: detección de matrículas, comunicación con la base de datos, gestión de zonas del parking y la interfaz del usuario.
6. **Pruebas:** Durante esta etapa, se adaptó la aplicación para asegurar su funcionamiento en diferentes tipos de pantallas y sistemas operativos, cumpliendo así con el objetivo inicial de lograr una interfaz multiplataforma.
7. **Memoria:** Junto con la implementación, se redactó la documentación del proyecto para facilitar la elaboración de la memoria final de forma estructurada y completa.
8. **Correcciones:** Finalmente, se revisaron el código y la memoria, aplicando mejoras y ajustes según las recomendaciones del tutor y los resultados de las pruebas.

3.2. Costes

Esta sección estima el coste que tendría el desarrollo del sistema si se llevara a cabo en un entorno real, teniendo en cuenta tanto el tiempo dedicado, como los recursos utilizados. El análisis contempla los costes de personal, materiales, despliegue y costes indirectos.

3.2.1. Coste de personal

El proyecto lo he realizado yo como único trabajador. Además, he contado con la supervisión y orientación de mi tutor. Las horas estimadas son:

- Análisis del problema, diseño y elección de tecnologías: 35 horas
- Estudio de tecnologías (Python, Flet, MongoDB): 25 horas
- Implementación (backend y frontend): 200 horas
- Pruebas y correcciones: 25 horas
- Redacción de la memoria: 80 horas

Total: **365 horas**.

Suponiendo un coste de 16 €/hora para un desarrollador junior:

- $365 \text{ horas} \times 16 \text{ €/hora} = \mathbf{5.840 \text{ €}}$

El tutor dedicó aproximadamente 1 hora semanal durante 12 semanas, a un coste estimado de 40 €/hora:

- $12 \text{ horas} \times 40 \text{ €/hora} = \mathbf{480 \text{ €}}$

Coste total de personal: 6.320 €

3.2.2. Coste de material

Durante el desarrollo del proyecto se utilizaron **tres teléfonos móviles antiguos**, reutilizados exclusivamente para realizar pruebas de detección de matrículas. Estos dispositivos permitieron simular la captura de imágenes sin necesidad de adquirir cámaras IP, abaratando bastante los costes durante la fase de desarrollo.

- Ordenador sobremesa (uso exclusivo durante 6 meses): 800 € de valor con vida útil de 4 años $\rightarrow 800 / 4 \times 0.5 = \mathbf{100 \text{ €}}$
- x3 Móviles reutilizados para pruebas: estimado **200 €**

Coste total de material durante el desarrollo: 300 €

En un entorno real, sería necesario sustituir estos móviles por **cámaras IP**, más adecuadas para un sistema de supervisión de plazas de parking. Los costes aproximados serían:

- x3 Cámaras IP Full HD (interior/exterior, visión nocturna): alrededor de 80 € cada una.

Coste futuro estimado para cámaras: $3 \times 80 \text{ €} = 240 \text{ €}$

Estos costes no los he incluido en el presupuesto principal, ya que serían para la fase de despliegue real.

3.2.3. Coste del despliegue

Para el almacenamiento de los datos del sistema, he utilizado **MongoDB Atlas**, una plataforma en la nube que permite desplegar bases de datos MongoDB con alta disponibilidad y escalabilidad.

Durante el desarrollo del proyecto se ha empleado el **plan gratuito (M0)**, que ofrece:

- Almacenamiento: hasta 512 MB.
- RAM: 1 GB
- vCPUs: compartidos
- Límite de conexiones simultáneas: hasta 500.
- Transferencia de datos semanal: hasta 10 GB de entrada y 10 GB de salida por semana.

Este plan ha sido suficiente para las necesidades del desarrollo y pruebas del sistema, por lo que **no he necesitado ningún coste durante la realización del proyecto**.

En caso de escalar el sistema a un entorno real con múltiples usuarios y más volumen de datos, sería necesario migrar a un plan de pago. MongoDB Atlas ofrece clústeres dedicados que se facturan por hora. La siguiente tabla resume algunos de los niveles más relevantes:

Tier	Almacenamiento	RAM	vCPUs	Precio/hora	Precio/mes aprox.
M10	10 GB	2 GB	2	\$0.08	\$57
M20	20 GB	4 GB	2	\$0.20	\$144
M30	40 GB	8 GB	2	\$0.54	\$388
M40	80 GB	16 GB	4	\$1.04	\$749

Tabla 3.1: Coste mensual aproximado de clústeres dedicados de MongoDB Atlas [5]

El precio mensual ha sido estimado en una media de 720 horas por mes.

3.2.4. Costes indirectos

Se estima un 10 % de los costes anteriores (personal + materiales + despliegue) en concepto de electricidad, conexión a internet y otros recursos.

$$10 \% \text{ de } (6,320 + 300) = \mathbf{662\text{€}}$$

3.2.5. Resumen de costes

Concepto	Coste (€)
Coste de personal	6.320
Coste de material	300
Despliegue	0
Costes indirectos	662
Total estimado	7.282

Tabla 3.2: Resumen de costes

4 | Análisis de requisitos

En este capítulo están los requisitos que tiene que cumplir el sistema de gestión del parking. Se distinguen los requisitos funcionales, no funcionales y de información.

4.1. Requisitos funcionales

Describen la interacción entre el sistema y su entorno, proporcionando servicios que proveerá el sistema o indicando la manera en que éste reaccionará ante determinados estímulos. Son aquellos que determinan qué debe hacer el software, los servicios que debe prestar y a que datos debe reaccionar [7].

- **RF-1.** El sistema debe capturar automáticamente los datos de cada vehículo, matrícula, fecha, tipo de vehículo y la zona donde aparca.
- **RF-2.** El sistema debe mostrar en tiempo real la ocupación de la *Zona 1* y la *Zona 2* en una pantalla y aplicación móvil.
- **RF-3.** Si la *Zona 2* está completa, el sistema debe indicar al conductor que se dirija a la *Zona 1* y viceversa.
- **RF-4.** El sistema debe almacenar la fecha de salida de cada vehículo para calcular tiempos de estancia y generar estadísticas.
- **RF-5.** El sistema debe permitir el registro de usuarios con diferentes roles (administrador, usuario).
- **RF-6.** El acceso a los datos y estadísticas debe estar protegido mediante inicio de sesión y control de permisos según el rol de cada usuario.
- **RF-7.** El sistema debe generar gráficos de uso (ocupación por zona, tipos de vehículos) y tabla de entradas por día.
- **RF-8.** El sistema tiene que tener mecanismos que manejen los casos de leer matrículas incorrectas o con errores.

4.2. Requisitos no funcionales

Describen cualidades o restricciones del sistema que no se relacionan de forma directa con el comportamiento funcional del mismo, son las condiciones que se imponen al sistema que no tienen que ver con la funcionalidad [7].

RNF-1: Rendimiento El tiempo de refresco de las plazas libres en pantalla será de 5 segundos.

RNF-2: Compatibilidad La aplicación deberá funcionar en Windows, Linux, macOS y en Android e iOS, adaptándose a cualquier tipo de pantalla.

RNF-3: Escalabilidad El sistema debe ser capaz de manejar un gran número de vehículos sin que afecte al rendimiento.

RNF-4: Usabilidad La interfaz debe ser sencilla y accesible para cualquier persona.

RNF-5: Seguridad Las contraseñas de los usuarios deben almacenarse de forma segura (hash + salt).

4.3. Requisitos de información

Describen necesidades de almacenamiento de información en el sistema así como la información que debe procesar el sistema.

■ RI-1: Vehículo

- Matrícula, tipo de vehículo, hora de entrada, hora de salida, zona asignada.
- Asociado a RF-1, RF-4, RF-7, RF-8.

■ RI-2: Estado del parking

- Numero de plazas totales y libres en cada zona, ocupación total.
- Asociado a RF-2, RF-3.

■ RI-3: Usuarios Registrados

- Identificador, Email, Contraseña, Rol.
- Asociado a RF-5, RF-6.

■ RI-4: Estadística de uso

- Tipos de vehículos, número de entradas/salidas, Vehículos por zona.
- Asociado a RF-7.

5 | Diseño de la aplicación

En este capítulo se describen las tecnologías utilizadas y las principales decisiones de diseño para el desarrollo del sistema. Se explican y justifican las herramientas usadas en función de los requisitos del sistema, también se explica el diseño de cada vista del frontend

5.1. Herramientas y tecnologías

1. **Python** Python es un lenguaje de programación ampliamente utilizado en las aplicaciones web, el desarrollo de software, la ciencia de datos y el machine learning (ML). Los desarrolladores utilizan Python porque es eficiente y fácil de aprender, además de que se puede ejecutar en muchas plataformas diferentes. [1]
2. **Flet** Flet es un framework que permite construir aplicaciones web, de escritorio y móviles en Python. Los controles Flet se basan en Flutter de Google. Flet añade su propio toque mediante la combinación de widgets más pequeños, la simplificación de complejidades, la aplicación de las mejores prácticas de interfaz de usuario. [3]
3. **Ultralytics** Ultralytics es una empresa especializada en el desarrollo de software para visión por ordenador basada en inteligencia artificial. Sus productos principales son:
 - **Ultralytics HUB**: Plataforma web sin necesidad de programación (no-code) donde te permite crear, entrenar y desplegar modelos de visión por ordenador a través de interfaces.
 - **Ultralytics YOLO (You Only Look Once)**: Arquitectura de redes neuronales optimizada para tareas de detección y reconocimiento de objetos. [15].
Gracias a esta tecnología, ha sido posible implementar un sistema de detección automática de vehículos y reconocimiento de matrículas en tiempo real.
4. **Flask** Flask es un framework ligero y flexible para el desarrollo web y aplicaciones web que utiliza Python. Flask es la opción preferida de los equipos de desarrollo para la creación de sitios web dinámicos, API y microservicios [2].
5. **MongoDB** MongoDB es una base de datos de documentos utilizada para crear aplicaciones de Internet altamente disponibles y escalables. Gracias a su esquema flexible, es muy popular entre los equipos de desarrollo que utilizan metodologías ágiles. [8].

6. **Visual Studio Code** Visual Studio Code (VS Code) es un editor de código fuente desarrollado por Microsoft, diseñado para ser ligero, rápido y altamente personalizable. VS Code se enfoca en ofrecer una experiencia ágil para la escritura y edición de código. [9].

5.2. Justificación de la elección tecnológica

5.2.1. ¿Por qué Python?

Uno de los motivos principales por los que decidí usar Python es su compatibilidad directa con los modelos de detección de vehículos y matrículas que utiliza el sistema. Herramientas como Ultralytics YOLO están diseñadas específicamente para funcionar en Python, por lo que utilizar otro lenguaje habría supuesto muchas más complicaciones con la detección.

Otro punto fuerte de Python es su integración con otras herramientas del proyecto. Por ejemplo, con Flask he podido construir rápidamente una API REST para comunicar el backend con la interfaz de usuario, y con pymongo me he conectado sin problemas a la base de datos MongoDB. Todo esto ha permitido mantener el desarrollo unificado dentro de un solo lenguaje, lo que simplifica mucho la arquitectura general de la aplicación.

Además, es un lenguaje muy sencillo de aprender y rápido para desarrollar, lo que me ha permitido centrarme más en la lógica del sistema y menos en problemas de sintaxis o configuración. Gracias a su flexibilidad, ha sido una opción muy práctica para implementar tanto la detección como la parte del servidor del proyecto.

5.2.2. ¿Por qué MongoDB?

Para el proyecto he elegido MongoDB como base de datos porque se adapta muy bien al sistema, siendo un entorno dinámico donde se registran constantemente datos de vehículos, usuarios y acciones en el parking. Aunque los datos que manejo tienen una estructura definida, como los registros de entrada y salida o la información de los vehículos detectados, MongoDB ofrece una forma más flexible y natural de trabajar con esta información frente a bases de datos relacionales más rígidas.

La principal ventaja de MongoDB en este caso es su modelo de datos orientado a documentos, que permite almacenar información en formato BSON (muy similar a JSON). Esto facilita que desde el backend, en Python, se puedan insertar, consultar y actualizar documentos de manera directa, sin necesidad de preocuparse por transformar objetos o adaptar consultas a un esquema complejo. Por ejemplo, si en el futuro quiero incluir un nuevo tipo de evento en los logs

o guardar información adicional sobre un vehículo detectado (como el color o la marca del vehículo), lo puedo hacer sin modificar toda la base de datos ni romper nada del sistema.

Otro punto fuerte es el rendimiento. MongoDB es capaz de manejar muchos registros en tiempo real sin que afecte la velocidad de respuesta, algo fundamental para este sistema que registra constantemente entradas, salidas o cambios de zona. Además, permite realizar búsquedas eficientes usando filtros, por ejemplo, buscar por matrícula exacta, u ordenar los resultados en base a un campo. En resumen, MongoDB no solo ha facilitado el desarrollo, sino que también garantiza que el sistema sea escalable y fácil de mantener a medida que crecen los datos o cambian los requisitos.

5.2.3. ¿Por qué Flet?

Al principio no tenía claro qué herramienta usar para la interfaz. Mi primera idea fue hacerla con React, ya que tenía algo de experiencia previa y me sentía cómodo con él. Sin embargo, después de comentarlo con mi tutor, me recomendó probar Flet.

Flet ofrece una forma muy práctica de construir interfaces modernas directamente desde Python, sin necesidad de usar tecnologías más complejas como HTML, CSS o JavaScript. Gracias a su enfoque declarativo y su integración directa con Python, he podido mantener todo el proyecto (tanto backend como frontend) dentro del mismo lenguaje, lo que ha hecho el desarrollo mucho más coherente.

También ha sido clave su capacidad multiplataforma. Flet permite ejecutar la misma aplicación tanto en entorno web como en diferentes sistemas operativos de escritorio sin modificar el código base, lo que me ha permitido probar su funcionamiento en distintos dispositivos y resoluciones de pantalla como se pedía desde un principio para este sistema.

5.2.4. ¿Por qué Flask?

Para el backend necesitaba montar apis para comunicar el sistema de detección con la interfaz. Entre todas las opciones disponibles, Flask me pareció la más adecuada.

Una de las principales razones por las que opté por Flask fue precisamente su facilidad para construir APIs REST. En pocas líneas puedes tener una ruta que devuelva datos en JSON, lo cual encajaba perfectamente con lo que necesitaba: exponer información como el número de plazas disponibles o los datos de los

vehículos capturados en el parking.

Además, Flask no obliga a seguir una estructura de proyecto compleja como otros frameworks más grandes (por ejemplo, Django), y eso me permitió tener un mayor control sobre cómo organizar las apis, servicios y la lógica de acceso a base de datos.

5.2.5. ¿Por qué Ultralytics?

Durante las primeras fases del desarrollo del sistema de detección de matrículas, comencé utilizando herramientas de reconocimiento óptico de caracteres (OCR) como **PyTesseract** y **EasyOCR**. Estas bibliotecas son muy utilizadas para extraer texto a partir de imágenes. Aunque estas librerías ofrecieron resultados decentes, no conseguían la precisión suficiente para un sistema en tiempo real de detección de matrículas.

A medida que avancé en el proyecto e integré **Ultralytics** para la detección de vehículos, descubrí la existencia de **Fast-ALPR**, una biblioteca especializada en reconocimiento automático de matrículas que también emplea modelos YOLO de Ultralytics. Fast-ALPR resultó ser una alternativa mucho más eficiente.

Fast-ALPR utiliza modelos como `yolo-v9-t-384-license-plate-end2end` para la detección de matrículas, y `global-plates-mobile-vit-v2-model` para el reconocimiento OCR, ofreciendo un rendimiento muy superior en términos de precisión y rapidez que las soluciones anteriores.

Para la detección de vehículos dentro del parking, utilicé directamente la librería **Ultralytics** con el modelo YOLOv8, concretamente el modelo preentrenado `yolov8n.pt`. Este modelo, proporciona un equilibrio entre precisión y velocidad, permitiendo identificar vehículos en tiempo real de forma eficiente y fiable. [6]

5.3. Diseño de la base de datos

La base de datos del sistema se ha diseñado utilizando MongoDB, una base de datos NoSQL orientada a documentos. Este enfoque permite una mayor flexibilidad frente a los esquemas rígidos de bases de datos relacionales. El almacenamiento se organiza en tres colecciones principales: `vehicles`, `logs` y `users`.

5.3.1. Colección `vehicles`

La colección `vehicles` almacena información sobre los vehículos detectados por el sistema. Cada documento representa una entrada individual con los siguientes campos principales:

- `_id`: Identificador único generado automáticamente por MongoDB.
- `plate`: Matrícula del vehículo detectado.
- `confidence`: Grado de confianza de la matrícula que ha detectado.
- `zona`: Zona del parking en la que se detectó (por ejemplo, "Zona 1" o "Zona 2").
- `date_in`: Fecha y hora de entrada del vehículo.
- `date_out`: Fecha y hora de salida (si ha salido).

5.3.2. Colección `logs`

La colección `logs` recoge todos los eventos importantes ocurridos en el sistema, tanto exitosos como fallidos. Cada documento contiene:

- `_id`: Identificador único del log.
- `action`: Tipo de acción registrada (por ejemplo, "Entrada" o "Error de cambio de zona").
- `description`: Descripción más detallada del evento.
- `plate`: Matrícula registrada en el evento.
- `zone`: Zona afectada por el evento.
- `timestamp`: Marca temporal que permite ordenar cronológicamente los eventos.

5.3.3. Colección users

La colección users contiene los datos de autenticación de los usuarios registrados. Los campos principales son:

- `_id`: Identificador único del usuario.
- `email`: Correo electrónico utilizado para iniciar sesión.
- `password`: Contraseña del usuario, almacenada de forma segura mediante un hash bcrypt.
- `admin`: Valor booleano que indica si el usuario tiene permisos de administrador.

La separación de los roles de usuario permite controlar el acceso a determinadas vistas o funcionalidades, asegurando que solo los administradores puedan ver estadísticas.

5.4. Interfaz del Usuario

5.4.1. Vista principal

Dado que la aplicación está orientada a mostrar información sobre el estado de las plazas de un parking, no fue necesario realizar un diseño especialmente complejo o elaborado. Dudé entre dos pantallas de inicio: por un lado, mostrar la estructura visual del parking con sus zonas diferenciadas, y por otro, utilizar algún widget como unas tarjetas con los datos esenciales. Al final, porque me gustaban ambas ideas, implementé las dos, pero en vistas separadas, accesibles mediante un botón. De esta forma, el usuario puede elegir entre la vista que muestra la estructura del parking o la que tiene la información en tarjetas.



Figura 5.1: Vista de Tarjetas.

Enlace a su implementación ->: [Implementación Tarjetas](#)



Figura 5.2: Vista de estructura.

Enlace a su implementación ->: [Implementación Estructura](#)

También he implementado un modo oscuro para la aplicación.



Figura 5.3: Vista en modo oscuro.

5.4.2. Vista del Panel del Administrador

Esta vista está dividida en dos secciones principales: registro de nuevas matrículas y gestión de las existentes.

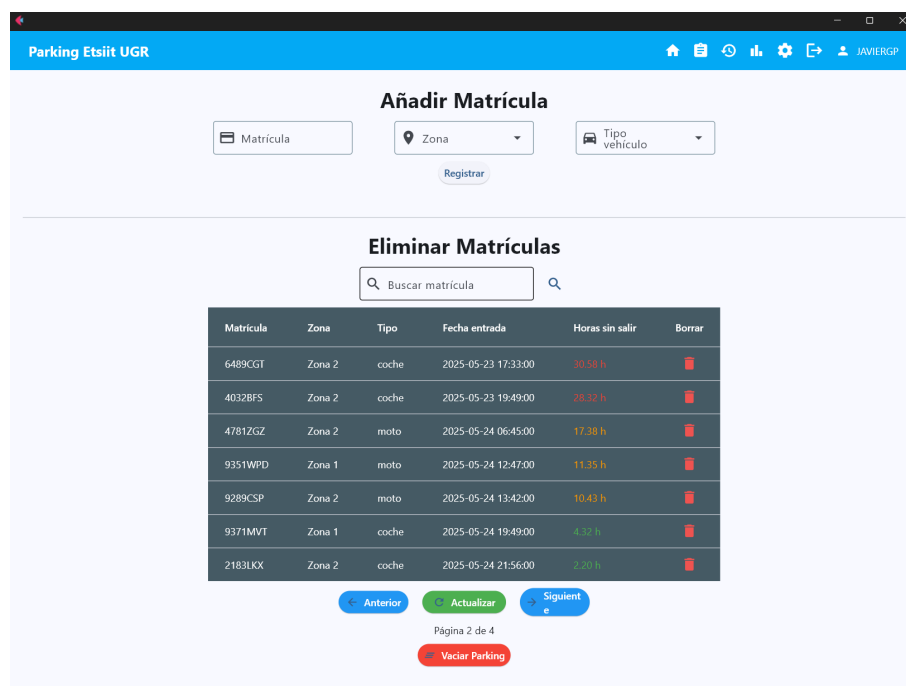


Figura 5.4: Vista del Panel de Administrador.

Enlace a su implementación ->: [Implementación Panel del Administrador](#)

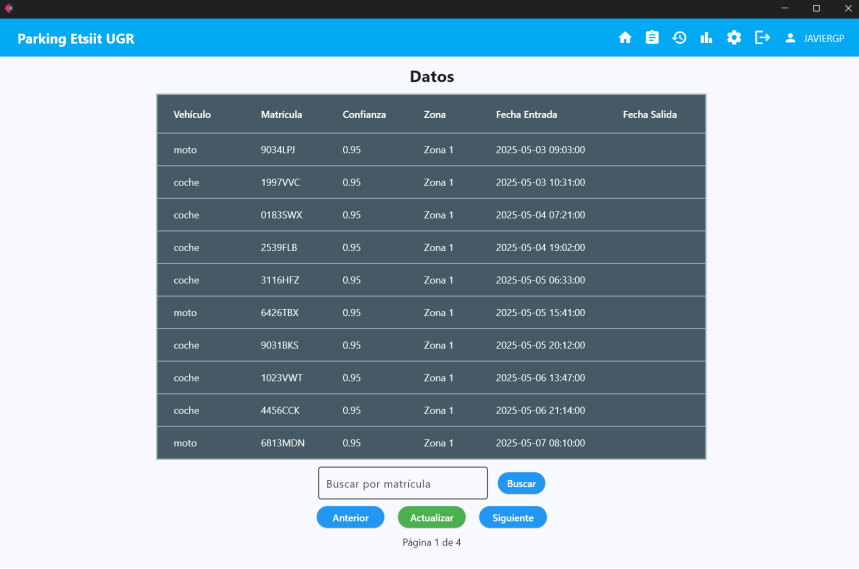
En la sección superior, implementé un formulario de registro de matrículas. La sección inferior permite eliminar matriculas manualmente en caso de algun error de detección con una tabla interactiva con todas las matrículas actualmente dentro del parking, destacando visualmente mediante un sistema de colores (verde, naranja y rojo) el tiempo de permanencia de cada vehículo. Esta característica permite identificar rápidamente aquellos vehículos que han superado determinados umbrales de tiempo (8 y 24 horas).

La interfaz incorpora funcionalidades de búsqueda por matrícula y un sistema de paginación. Además, incluye diálogos de confirmación para operaciones de borrado. También se adapta automáticamente a diferentes tamaños de pantalla, reorganizando sus elementos.

5.4.3. Vistas de Datos

En cuanto a la visualización de los datos registrados, tanto de los vehículos que han accedido al parking como de los logs de actividad, diseñé una vista basada en tablas.

Para el caso de la vista de datos, incluí funcionalidades para ordenar los datos por cualquiera de sus campos (fecha de entrada, fecha de salida, zona, matrícula, etc.), lo que facilita un análisis rápido de la información. Además, añadí un campo de búsqueda que permite filtrar directamente por una matrícula concreta. Para ambas tablas, tanto la de vehículos como la de logs, implementé paginación con el objetivo de manejar grandes volúmenes de datos.



Vehículo	Matrícula	Confianza	Zona	Fecha Entrada	Fecha Salida
moto	9034LPJ	0.95	Zona 1	2025-05-03 09:03:00	
coche	1997VVC	0.95	Zona 1	2025-05-03 10:31:00	
coche	0183SWX	0.95	Zona 1	2025-05-04 07:21:00	
coche	2539FLB	0.95	Zona 1	2025-05-04 19:02:00	
coche	3116HFZ	0.95	Zona 1	2025-05-05 06:33:00	
moto	6426TBX	0.95	Zona 1	2025-05-05 15:41:00	
coche	9031BKS	0.95	Zona 1	2025-05-05 20:12:00	
coche	1023VWT	0.95	Zona 1	2025-05-06 13:47:00	
coche	4456CCK	0.95	Zona 1	2025-05-06 21:14:00	
moto	6813MDN	0.95	Zona 1	2025-05-07 08:10:00	

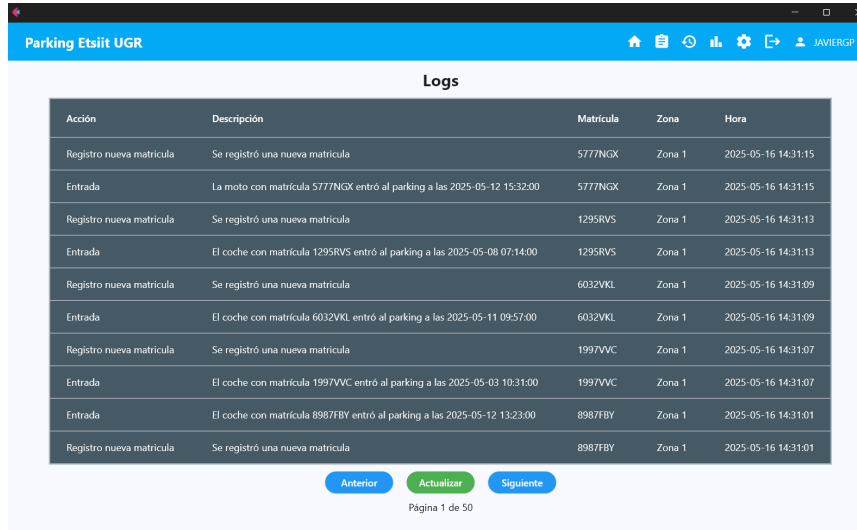
Buscar por matrícula

Página 1 de 4

Figura 5.5: Vista de Datos.

Enlace a su implementación ->: [Implementación Datos](#)

Logs:



Acción	Descripción	Matricula	Zona	Hora
Registro nueva matricula	Se registró una nueva matricula	5777NGX	Zona 1	2025-05-16 14:31:15
Entrada	La moto con matrícula 5777NGX entró al parking a las 2025-05-12 15:32:00	5777NGX	Zona 1	2025-05-16 14:31:15
Registro nueva matricula	Se registró una nueva matricula	1295RVS	Zona 1	2025-05-16 14:31:13
Entrada	El coche con matrícula 1295RVS entró al parking a las 2025-05-08 07:14:00	1295RVS	Zona 1	2025-05-16 14:31:13
Registro nueva matricula	Se registró una nueva matricula	6032VKL	Zona 1	2025-05-16 14:31:09
Entrada	El coche con matrícula 6032VKL entró al parking a las 2025-05-11 09:57:00	6032VKL	Zona 1	2025-05-16 14:31:09
Registro nueva matricula	Se registró una nueva matricula	1997VVC	Zona 1	2025-05-16 14:31:07
Entrada	El coche con matrícula 1997VVC entró al parking a las 2025-05-03 10:31:00	1997VVC	Zona 1	2025-05-16 14:31:07
Entrada	El coche con matrícula 8987FBY entró al parking a las 2025-05-12 13:23:00	8987FBY	Zona 1	2025-05-16 14:31:01
Registro nueva matricula	Se registró una nueva matricula	8987FBY	Zona 1	2025-05-16 14:31:01

Anterior Actualizar Siguiente

Página 1 de 50

Figura 5.6: Vista de Logs.

Enlace a su implementación ->: [Implementación Logs](#)

También desarrollé una vista pensada específicamente para mostrarla en una pantalla en la entrada del parking. Esta interfaz ofrece, de un vistazo, la información más relevante sobre la disponibilidad de plazas en cada zona, facilitando a los conductores la toma de decisiones antes de acceder, en caso de que no puedan usar la aplicación móvil o se les olvide hacerlo.



Figura 5.7: Pantalla de entrada.

Enlace a su implementación ->: [Implementación pantalla de entrada](#)

5.4.4. Gráficas y Estadísticas

Por último, diseñé una sección de gráficas que ofrece una visión estadística del uso del parking. Esta vista incluye tres gráficas: un gráfico circular que muestra el porcentaje de tipos de vehículos que han accedido (coches o motos), un gráfico de barras con la ocupación por zonas, y un gráfico de líneas que representa el número de entradas al parking por día.



Figura 5.8: Vista de gráficas.

Enlace a su implementación ->: [Implementación Gráficas](#)

5.4.5. Navegación y Accesibilidad

Es importante mencionar que las vistas de gráficas y las tablas con los datos de los vehículos y logs están restringidas a los administradores del sistema. Los usuarios normales pueden acceder solo a las vistas básicas, como la visualización de la estructura del parking o las tarjetas informativas de las plazas disponibles, siempre que estén antes logueados.

En todas las vistas se ha incluido una barra de navegación. Esta se adapta al ancho de la pantalla para que en pantallas pequeñas todos los ítems se vean desde un desplegable, y en pantallas normales se ven directamente los iconos de las diferentes vistas.



Figura 5.9: Barra de navegación.

Pantallas pequeñas:

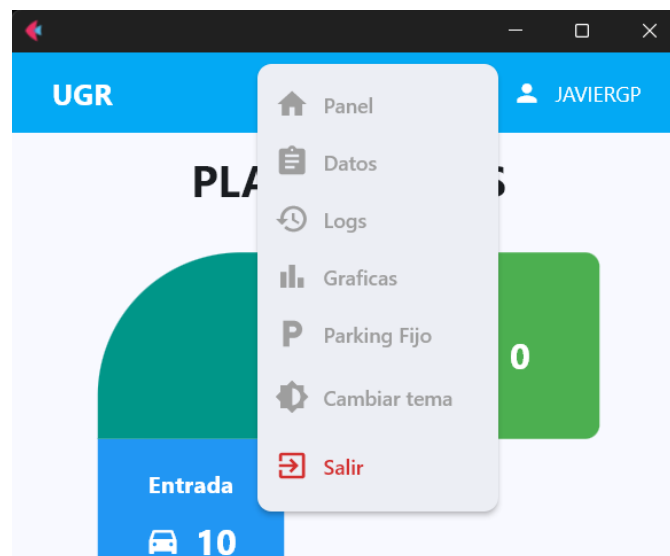


Figura 5.10: Barra de navegación móvil.

Enlace a su implementación ->: [*Implementación Barra de Navegación*](#)

5.4.6. Autenticación de Usuarios

Para la autenticación de usuarios, como la aplicación está pensada para la universidad, quise hacer algo igual, por lo que me inspiré en el diseño de la página del login del correo institucional de la universidad. Así, los usuarios se sentirán cómodos al reconocer el estilo y la estructura.

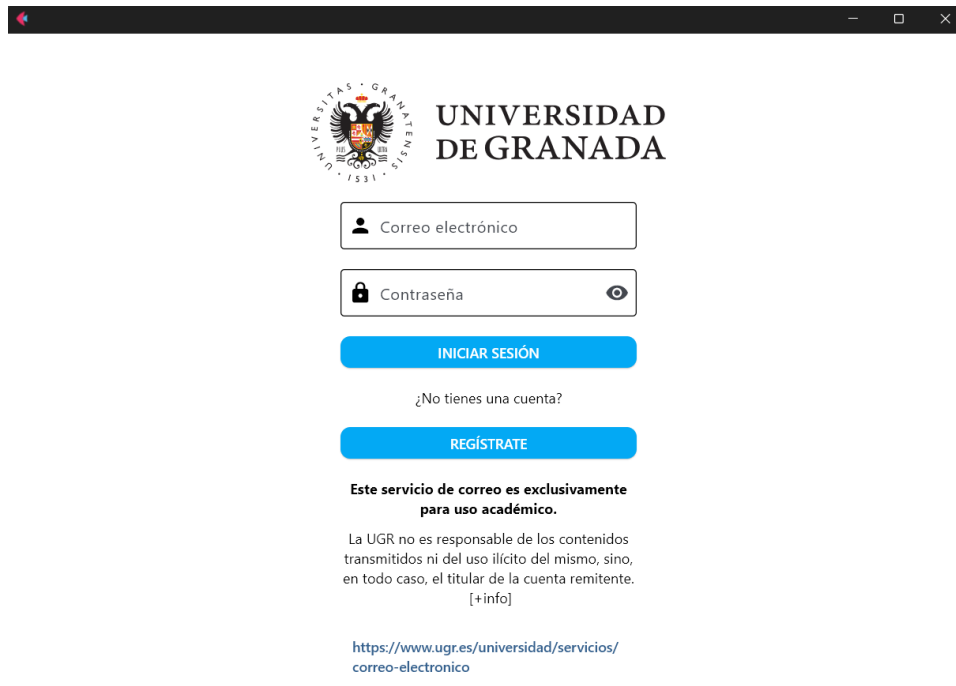
En una implementación real, lo ideal sería que la autenticación se integrase con el mismo sistema que se utiliza en la UGR, de modo que el personal autorizado pudiera acceder con sus credenciales institucionales. Esto evitaría la necesidad de crear nuevas cuentas o recordar más contraseñas. Dicha integración facilitaría la gestión de permisos y reforzaría la seguridad del sistema.

Para mostrar la imagen en la aplicación, la he almacenado en una carpeta de mi repositorio de GitHub, lo que permite acceder a ella directamente gracias a la funcionalidad de GitHub que proporciona el contenido binario de los archivos. Al utilizar la URL de GitHub del contenido raw, se obtiene un enlace directo al archivo, lo que permite renderizar la imagen desde cualquier navegador o aplicación.

Por ejemplo, el enlace usado en la aplicación es:

```
https://raw.githubusercontent.com/jav1ergp/TFG/main/  
backend/images/ugr-horizontal-color.svg
```

La vista de login solicita el correo electrónico y la contraseña. Estos son los únicos datos que se necesitan para autenticar al usuario.



UNIVERSIDAD DE GRANADA

Correo electrónico

Contraseña

INICIAR SESIÓN

¿No tienes una cuenta?

REGÍSTRATE

Este servicio de correo es exclusivamente para uso académico.

La UGR no es responsable de los contenidos transmitidos ni del uso ilícito del mismo, sino, en todo caso, el titular de la cuenta remitente.
[+info]

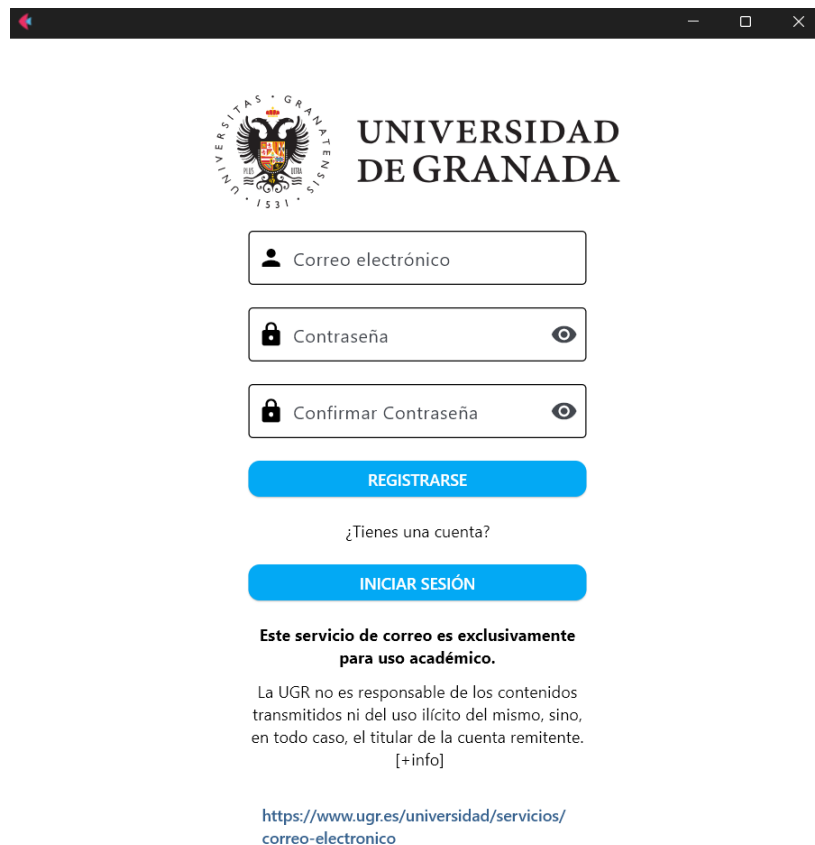
<https://www.ugr.es/universidad/servicios/correo-electronico>

Figura 5.11: Login.

Enlace a su implementación ->: [Implementación Login](#)

Diagrama de Secuencia ->: [Diagrama de Secuencia Autenticación](#)

La vista de registro incluye algunas validaciones, el formulario solicita el correo, asegurándose de que este tenga un formato válido y que no esté ya registrado antes en la base de datos. Además, el usuario debe crear una contraseña, e ingresarla dos veces para asegurarse de que no haya errores.



The screenshot shows a web browser window with the University of Granada logo and name at the top. Below the logo, there are three input fields for registration: 'Correo electrónico' (Email), 'Contraseña' (Password), and 'Confirmar Contraseña' (Confirm Password). Each field has a corresponding icon (person, lock, and lock with eye) and a toggle eye icon for password visibility. Below these fields are two blue buttons: 'REGISTRARSE' (Register) and '¿Tienes una cuenta?' (Do you have an account?). Below the buttons is a section titled 'Este servicio de correo es exclusivamente para uso académico.' (This email service is exclusively for academic use.) followed by a disclaimer: 'La UGR no es responsable de los contenidos transmitidos ni del uso ilícito del mismo, sino, en todo caso, el titular de la cuenta remitente.' (UGR is not responsible for the contents transmitted or the illegal use of the same, but, in any case, the holder of the sending account.) and a link '[+info]'. At the bottom, there is a URL: <https://www.ugr.es/universidad/servicios/correo-electronico>.

Figura 5.12: Registro.

Enlace a su implementación ->: [Implementación Registro](#)

Diagrama de Secuencia ->: [Diagrama de Secuencia Autenticación](#)

6 | Implementación

6.1. Estructura de archivos y preparación del entorno

El proyecto ha sido estructurado en distintos módulos para facilitar su desarrollo, mantenimiento y escalabilidad. La organización de las carpetas y archivos son:

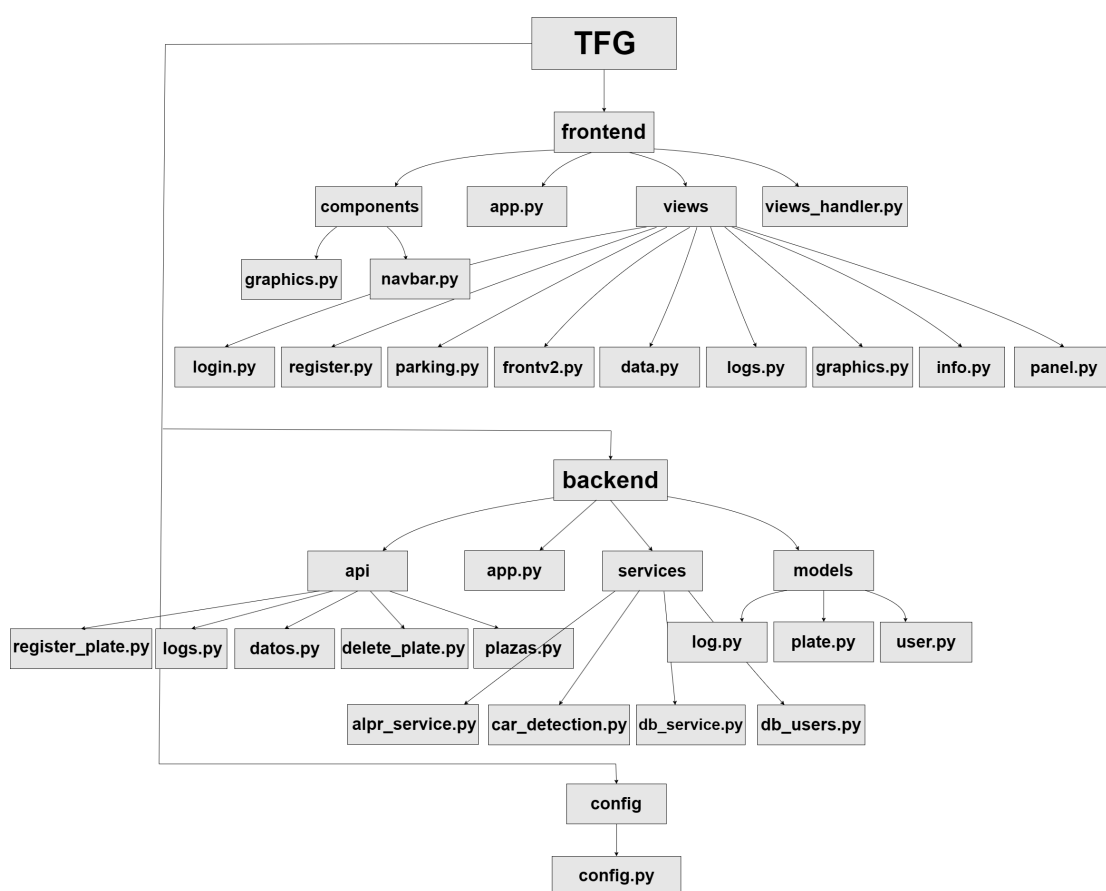


Figura 6.1: Estructura de archivos del proyecto

La carpeta principal TFG contiene tanto el código del frontend como del backend, así como los archivos de configuración y dependencias necesarias para la ejecución. A continuación, se describen las carpetas principales:

- **frontend/**: Contiene la interfaz gráfica desarrollada en Flet. Dentro de esta carpeta se encuentran las vistas (`views/`), los componentes reutilizables

como el navbar o las gráficas (`components/`), y el archivo `app.py` que es el ejecutable de la aplicación.

- **backend/**: Es la lógica y el procesamiento de datos del servidor . Se divide en:
 - `api/`: Define las rutas de la API mediante Flask.
 - `models/`: Contiene las clases y estructuras de datos utilizadas en el sistema.
 - `services/`: Incluye los módulos encargados de la lógica del sistema, como el acceso a la base de datos o la detección de matrículas.
 - `app.py`: Archivo principal para iniciar el servidor Flask.
- **config/**: Contiene los archivos de configuración en variables de entorno.

6.1.1. Preparación del entorno

Para el desarrollo del sistema se ha utilizado el entorno de desarrollo integrado (IDE) Visual Studio Code. Esta elección es porque es la herramienta que más he utilizado a lo largo de la carrera y con la que me siento más cómodo trabajando. Además, ofrece muchas extensiones útiles para el desarrollo en Python, así como una buena integración con sistemas de control de versiones.

Durante el desarrollo, todo el proyecto ha sido gestionado y almacenado en un repositorio en mi cuenta personal de GitHub.

Las dependencias necesarias para este proyecto están en el archivo `requirements.txt`, donde se especifican todas las bibliotecas necesarias tanto para el frontend como para el backend del sistema. A continuación, se detallan las principales librerías utilizadas, clasificadas por su funcionalidad dentro del sistema:

- **Framework web:**
 - `flask==3.0.3`
Utilizado para construir la API RESTful que permite la comunicación entre el cliente y el servidor.
- **Interfaz gráfica (UI):**
 - `flet==0.27.6`
Framework en Python utilizado para desarrollar la interfaz gráfica de usuario del sistema de gestión del parking.

- **Base de datos:**

- `pymongo==4.10.1`
Biblioteca que permite la interacción entre la aplicación en Python y la base de datos MongoDB.

- **Visión por computador y reconocimiento de matrículas:**

- `opencv-python==4.10.0.84`
Utilizado para el procesamiento de imágenes capturadas por las cámaras.
- `ultralytics==8.3.3`
Framework que facilita el uso de modelos YOLO para la detección de vehículos.
- `fast-alpr==0.1.2`
Biblioteca para el reconocimiento automático de matrículas (ALPR).

- **Comunicación HTTP:**

- `requests==2.32.3`
Librería utilizada para realizar peticiones HTTP síncronas.
- `aiohttp==3.10.7`
Permite realizar peticiones HTTP de forma asíncrona.

- **Utilidades adicionales:**

- `datetime==5.5`
Módulo para el manejo de fechas y horas.
- `asyncio==3.4.3`
Biblioteca para programación asíncrona y gestión de tareas concurrentes.

Antes de ejecutar el sistema, es necesario instalar todas estas dependencias. Esto se hace desde la terminal con el siguiente comando:

```
pip install -r requirements.txt
```

6.2. Arquitectura general del sistema

En esta sección voy a explicar cómo se comunican los diferentes componentes del sistema de gestión del parking, desde que un vehículo entra hasta que la información se muestra en la interfaz de usuario.

La Figura 6.2 muestra un diagrama general de esta arquitectura, donde se representan los módulos principales y el flujo de datos entre ellos.

6.2.1. Diagrama de Arquitectura

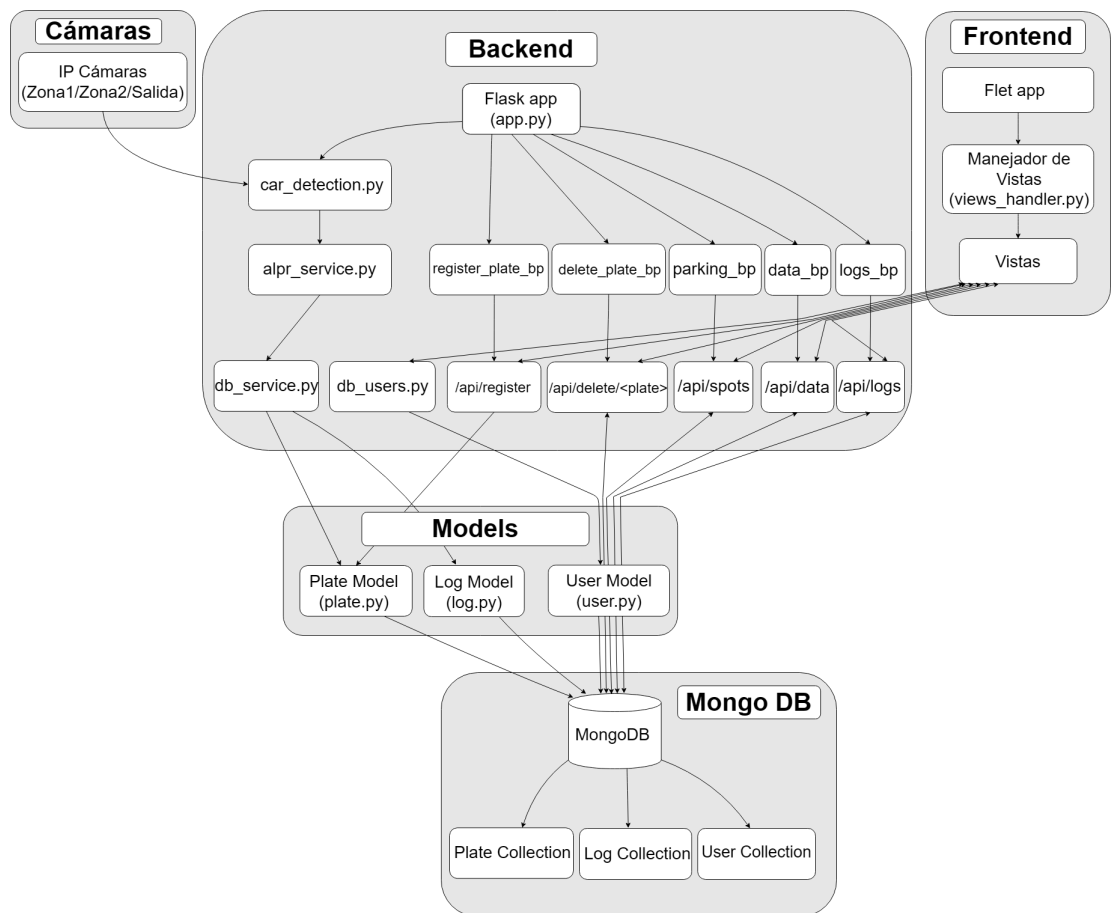


Figura 6.2: Sistema

6.2.2. Flujo de Datos Principal

Voy a explicar los pasos que sigue el sistema, desde que un vehículo entra en el parking hasta que su información se muestra en la interfaz, se componen de varios pasos clave:

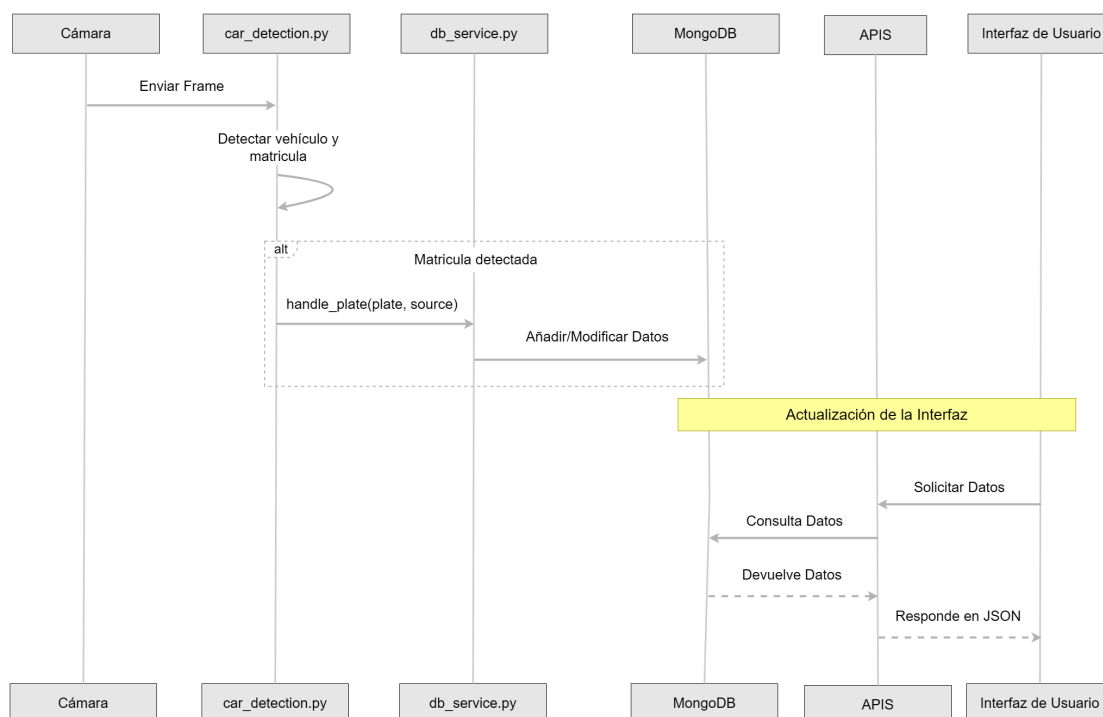


Figura 6.3: Diagrama de Secuencia del Sistema(simplificado)

En primer lugar, las cámaras IP instaladas en distintos puntos del aparcamiento (como la entrada, una zona intermedia y la salida) capturan imágenes continuamente. Estas imágenes son procesadas por el sistema, que detecta la presencia de vehículos y reconoce sus matrículas utilizando los modelos de visión por computador.

Una vez reconocida la matrícula, la información del vehículo (como el tipo, la matrícula, la hora de entrada o salida y la zona correspondiente) se almacena en la base de datos MongoDB. Después, el frontend pide los datos almacenados en mongo a través de APIs RESTful.

Finalmente, la interfaz de usuario muestra estos datos en tiempo real, permitiendo a los usuarios y administradores ver toda la información sobre el parking.

6.3. Backend

El backend del sistema de gestión de parking está construido con Flask y se encarga de procesar la detección de vehículos, gestionar los datos y proporcionar APIs mediante *blueprints* para la interfaz de usuario.

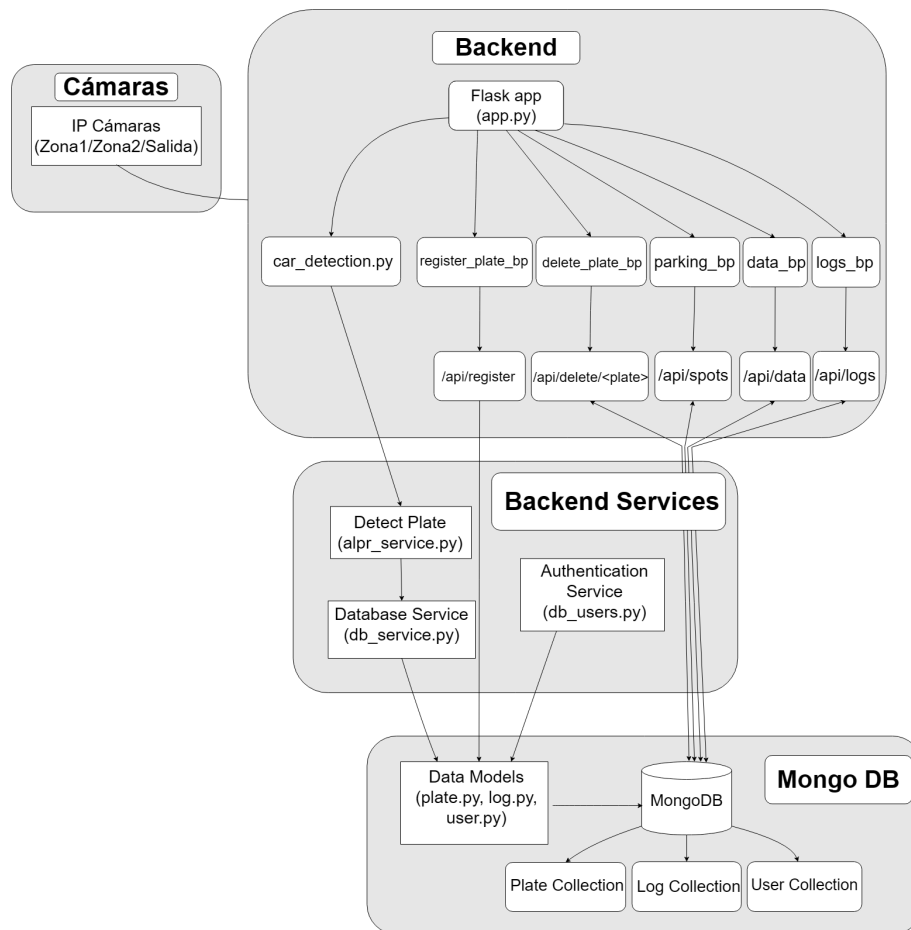


Figura 6.4: Backend

6.3.1. Inicialización del Sistema

El archivo `app.py` es el punto de entrada principal de la aplicación. Aquí se ejecuta el servidor Flask, registrándose los blueprints y lanzando el proceso de detección de matrículas.

Código 6.1: Archivo backend/app.py

```
1 import sys
2 from pathlib import Path
3
4 PROJECT_ROOT = Path(__file__).parent.parent
5 sys.path.append(str(PROJECT_ROOT))
6
7 from flask import Flask
8 from backend.api import plazas, datos, logs
9 from backend.services.car_detection import
   start_detection
10 from config.back_config import URL_ENTRADA, URL_ZONA,
   URL_SALIDA
11
12 app = Flask(__name__)
13
14 # Registrar rutas
15 app.register_blueprint(plazas.parking_bp)
16 app.register_blueprint(datos.data_bp)
17 app.register_blueprint(logs.logs_bp)
18
19 # Iniciar deteccion automatica
20 start_detection(URL_ENTRADA, URL_ZONA, URL_SALIDA)
21
22 if __name__ == "__main__":
23     app.run(host='0.0.0.0', port=5000)
```

6.3.2. API

Las rutas definen los endpoints de la API del sistema de gestión del parking. Cada una tiene funcionalidades específicas que permiten consultar o interactuar con los datos en la base de datos.

6.3.2.1. API de Plazas

La API de plazas de parking ofrece información en tiempo real sobre la disponibilidad de plazas en el sistema de gestión del parking. Permite conocer cuántas plazas están ocupadas y cuántas están libres, diferenciando entre coches y motos, y también entre las zonas de entrada y salida. Esta información se usa para mostrar el estado actual de las plazas en la interfaz de usuario.

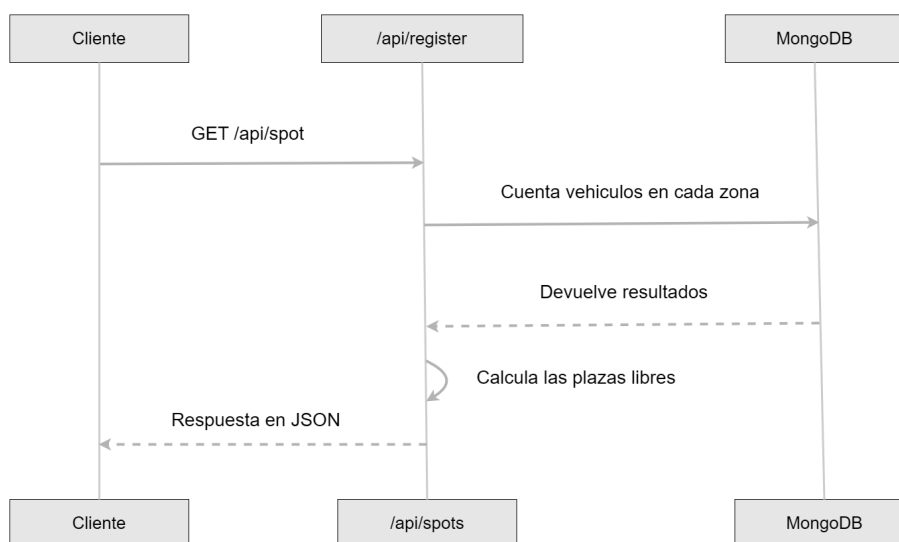


Figura 6.5: Diagrama de Secuencia API Plazas

El endpoint de esta API es /api/spots, que responde a peticiones GET con un objeto JSON que incluye los datos de ocupación y plazas disponibles. No necesita recibir parámetros y devuelve la cantidad de plazas ocupadas y libres para cada tipo de vehículo y zona. Estos resultados se obtienen restando la capacidad total de cada zona al número de vehículos detectados de cada zona.

6.3.2.2. API de Datos

La API de datos ofrece acceso a la información de los vehículos registrados en la base de datos. Esta API permite al frontend obtener registros de vehículos con opciones para filtrar, ordenar y paginar los resultados, facilitando así la visualización de la información en la interfaz de usuario.

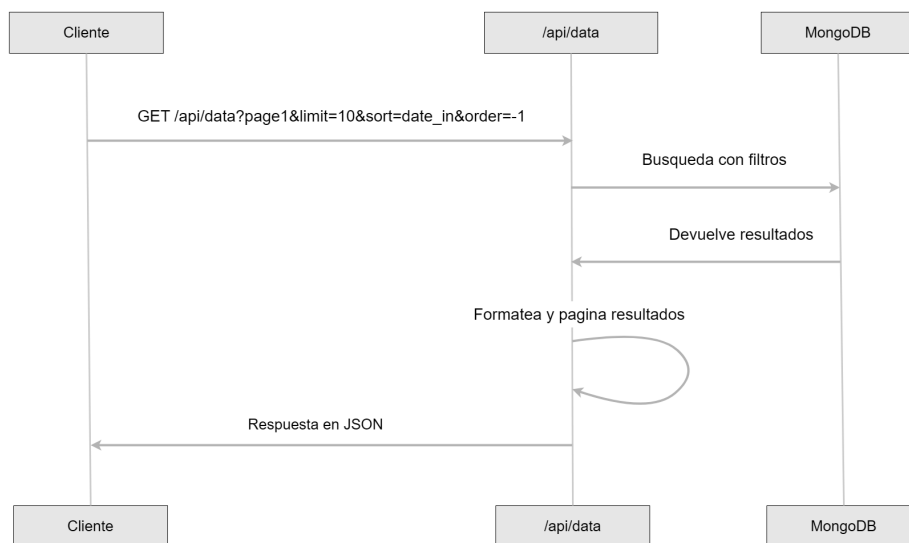


Figura 6.6: Diagrama de Secuencia API Datos

El endpoint disponible es `/api/data`, que acepta peticiones GET con varios parámetros como página, límite de registros por página, campo de ordenación, sentido del orden y búsqueda por matrícula. La respuesta es un objeto JSON que contiene la lista de vehículos con los filtros, la información sobre el total de registros y la paginación.

La implementación se basa en un *blueprint* de Flask que realiza consultas a la colección `vehicles` de MongoDB. La API procesa los parámetros de la petición para construir una consulta que filtra por matrícula si se indica, ordena por el campo y sentido especificados, y aplica paginación usando los métodos `skip()` y `limit()`. Además, gestiona valores por defecto, por ejemplo, para la fecha de salida cuando el vehículo aún no ha salido.

6.3.2.3. API de Logs

La API de logs proporciona acceso a los registros de actividad del sistema. Estos logs recogen eventos relevantes, como entradas y salidas de vehículos o cambios de zona, permitiendo a los administradores hacer un seguimiento de las operaciones. Además, el sistema guarda logs incluso si se producen fallos en las operaciones.

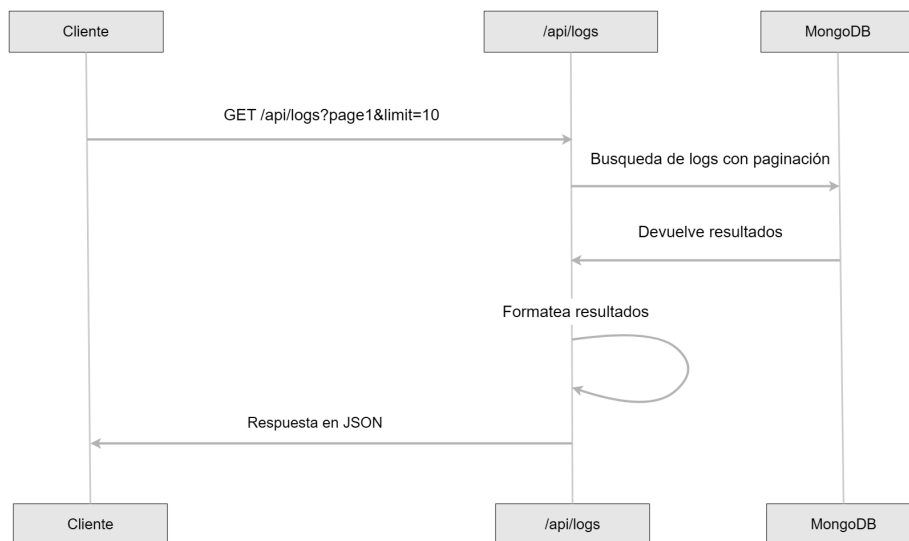


Figura 6.7: Diagrama de Secuencia API Logs

El endpoint es `/api/logs`, que responde a solicitudes GET y devuelve una lista paginada de los registros ordenados cronológicamente, mostrando primero los eventos más recientes. Los parámetros disponibles para la paginación son el número de página y la cantidad de registros por página, siendo sus valores por defecto 1 y 10.

La implementación se realiza mediante un *blueprint* de Flask que consulta la colección `logs` de MongoDB. La API procesa los parámetros de paginación para calcular el desplazamiento y el límite de documentos. Después, recupera los registros ordenados por fecha y hora (`timestamp`) y devuelve los datos en formato JSON.

6.3.2.4. API de Registro de Vehículos

La API de registro permite almacenar nuevos registros de vehículos que acceden al sistema. Este mecanismo es útil para gestionar casos en los que las cámaras no han detectado correctamente la matrícula o si algún vehículo ya estaba presente antes de poner en funcionamiento el sistema.

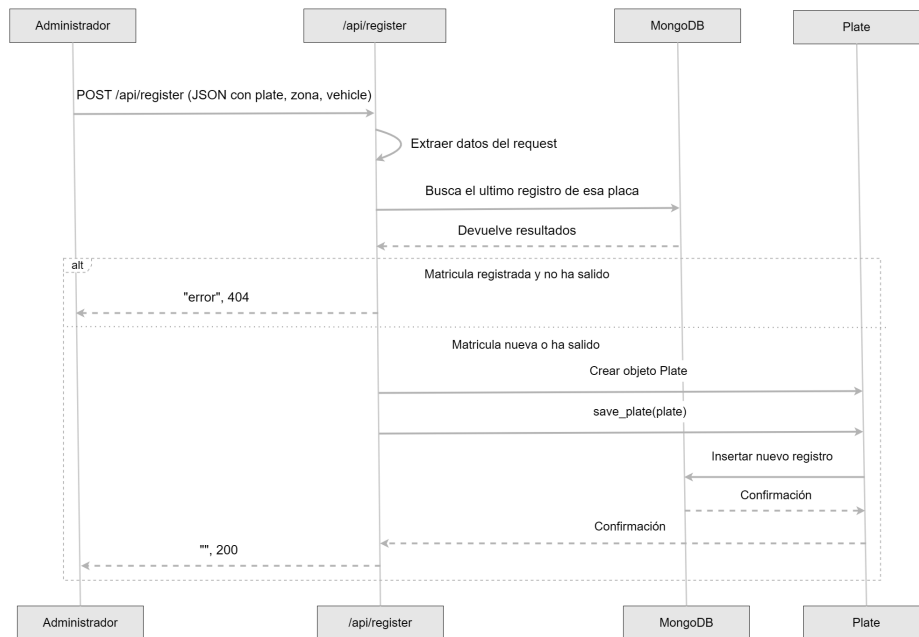


Figura 6.8: Diagrama de Secuencia API Registro

Su endpoint es /api/register, acepta solicitudes de tipo POST. El cuerpo de la solicitud debe contener un objeto JSON con los datos del vehículo. Al recibir la solicitud, el sistema extrae la matrícula, la zona y el tipo de vehículo. A partir de esta información se crea un objeto del modelo Plate, con fecha actual(date_in) y una confianza del 100%.

Este objeto se guarda en la base de datos, mediante el método save_plate() del modelo Plate. La API responde con un código HTTP 200 si el registro se hizo bien, si encuentra la misma matrícula dentro del parking, devuelve error y un código HTTP 404.

6.3.2.5. API de Eliminación de Vehículos

La API de eliminación permite borrar registros de vehículos dentro del sistema. Esta funcionalidad es útil tanto para la eliminación manual de vehículos concretos, o un vaciado general en que hayan muchos vehículos incorrectos dentro del parking.

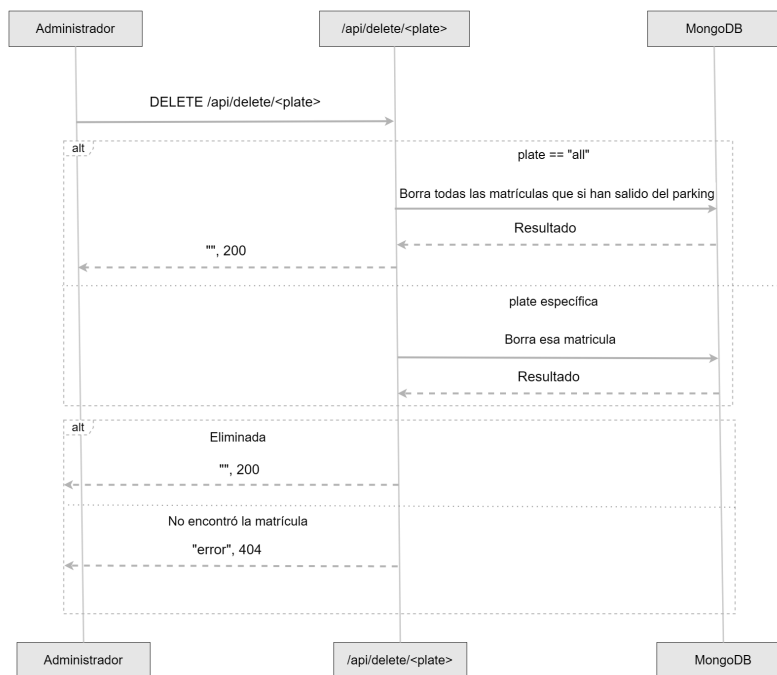


Figura 6.9: Diagrama de Secuencia API Eliminación

El endpoint es `/api/delete/<plate>`, acepta solicitudes de tipo DELETE y permite dos modos de uso:

Si se proporciona una matrícula específica (por ejemplo, `/api/delete/1234ABC`), se intentará eliminar el registro correspondiente a esa matrícula.

Si se utiliza el valor especial `all` (`/api/delete/all`), se eliminarán todos los registros de vehículos que estén dentro del parking y no hayan salido (es decir, aquellos cuyo campo `date_out` esté a `null`).

Para el caso de una matrícula concreta, se utiliza el método `delete_one()`, mientras que para el caso global se emplea `delete_many()` con un filtro que busca los registros con campo `date_out = null`. El servidor responde con un código HTTP 200 si la operación fue bien o con 404 si hubo algún error.

6.3.3. Modelos

Los modelos en el sistema de gestión de parking definen la estructura de los datos y proporcionan métodos para interactuar con la base de datos.

6.3.3.1. Modelo Plate

Este modelo representa un vehículo con su matrícula y datos asociados.

Atributos principales:

- `plate_text`: Matrícula del vehículo detectado.
- `confidence`: Nivel de confianza del reconocimiento de la matrícula (valor numérico).
- `vehicle`: Tipo de vehículo detectado (por ejemplo, coche, moto).
- `zona`: Zona del parking en la que ha sido detectado el vehículo.
- `date_in`: Fecha y hora de entrada del vehículo al parking.
- `date_out`: Fecha y hora de salida del vehículo del parking (si ha salido).

Métodos principales:

- `save_plate_to_db`: Guarda un nuevo registro de vehículo en la base de datos y genera automáticamente un log de entrada.
- `is_valid_plate`: Valida el formato de la matrícula, aceptando tanto estilos modernos como antiguos.

6.3.3.2. Modelo Log

Este modelo representa un registro de actividad dentro del sistema, como entradas y salidas de vehículos.

Atributos principales:

- `action`: Tipo de acción realizada (por ejemplo, entrar, salir, errores).
- `description`: Descripción del evento registrado.
- `plate`: Matrícula del vehículo involucrado en la acción.
- `zona`: Zona del parking donde se produjo la acción.
- `timestamp`: Fecha y hora exacta del evento.

Métodos principales:

- `save_log`: Guarda un nuevo registro de actividad en la base de datos.

6.3.3.3. Modelo User

El modelo User representa a los usuarios del sistema y gestiona la autenticación.

Atributos principales:

- email: Identificador único del usuario.
- password: Contraseña hasheada del usuario.
- admin: Boolean que indica si el usuario tiene permisos de administrador.

Métodos principales:

- create: Crea un nuevo usuario con la contraseña hasheada usando bcrypt.
- check_password: Verifica si una contraseña en texto plano coincide con el hash almacenado.
- is_valid_email: Valida el formato de un correo electrónico.

6.3.4. Servicios

Los servicios del sistema de gestión del parking se encargan de implementar la lógica del sistema y de actuar como capa intermedia entre las rutas, los modelos y la base de datos.

6.3.4.1. Servicio de Detección de Vehículos

Para supervisar en tiempo real la entrada, zona interior y salida del parking, se utiliza una función que inicia el procesamiento de cada cámara en paralelo mediante hilos (threads). Esto permite detectar vehículos simultáneamente en las diferentes cámaras del sistema sin bloquear la ejecución.

Código 6.2: Archivo car_detection.py

```
1 def start_detection(url_entrada, url_zona, url_salida):
2     cameras = {
3         "Entrada": url_entrada,
4         "Zona": url_zona,
5         "Salida": url_salida
6     }
7
8     for source, url in cameras.items():
9         thread = Thread(target=process_camera, args=(
10             source, url))
11         thread.daemon = True
12         thread.start()
```

La función `process_camera` se encarga de procesar la imagen recibida desde una cámara IP en tiempo real. Utiliza el modelo YOLOv8 para detectar vehículos y, si se identifica uno, se intenta identificar su matrícula y procesarla.

Código 6.3: Archivo `backend/services/car_detection.py`

```
1 def process_camera(source, url):
2     model = YOLO('yolov8n.pt')
3     cap = cv2.VideoCapture(url)
4
5     if not cap.isOpened():
6         print(f"No se pudo acceder a la camara ({url}).")
7         return
8
9     while True:
10        ret, frame = cap.read()
11        if not ret:
12            print(f"Error al capturar frame de la camara {source}.")
13            break
14
15        results = model(frame)
16
17        # Procesar cada deteccion
18        for result in results[0].boxes:
19            confidence = float(result.conf)
20            if result.cls in [2, 3] and confidence >= 0.85:
21                class_id = int(result.cls)
22
23                if class_id == 2:
24                    vehicle = "coche"
25                elif class_id == 3:
26                    vehicle = "moto"
27
28                plate = alpr_service.detect_plate(frame, vehicle)
29
30                if plate is not None:
31                    db_service.handle_plate(plate, source)
32
33        cap.release()
```

Diagrama de Flujo →: [*Diagrama de flujo del proceso de detección*](#)

6.3.4.2. Servicio de Reconocimiento de Matrículas

Esta función se encarga de detectar matrículas a partir de un fotograma utilizando la biblioteca Fast-ALPR. Si el modelo detecta texto válido con un nivel de confianza superior al 95%, se crea un objeto Plate con los datos obtenidos (matrícula, tipo de vehículo, zona y hora). Si no se detecta una matrícula válida, la función devuelve None.

Código 6.4: Archivo backend/services/alpr_service.py

```
1 def detect_plate(frame, vehicle):
2     alpr = init_alpr()
3     results = alpr.predict(frame)
4
5     if results and results[0].ocr.text != " ":
6         ocr_result = results[0].ocr
7         license_plate_text = ocr_result.text
8         confidence = round(ocr_result.confidence, 2)
9         date_in = datetime.now().strftime("%Y-%m-%d %H:%M
10             :%S")
11         zona = "entrada"
12
13         if Plate.is_valid_plate(license_plate_text) and
14             confidence > 0.95:
15             return Plate(license_plate_text, confidence,
16                 vehicle, date_in, zona)
17
18     return None
```

Enlace a Diagrama de Flujo ->: [Diagrama de flujo del proceso de detección](#)

6.3.4.3. Servicio de Procesamiento de Matriculas y Logs

Toda la lógica relacionada con la gestión de matrículas y logs en su almacenamiento en MongoDB se implementa en el archivo `db_service.py` en la función `handle_plate(plate,source)`. La lógica del procesamiento se ve en el siguiente diagrama de flujo:

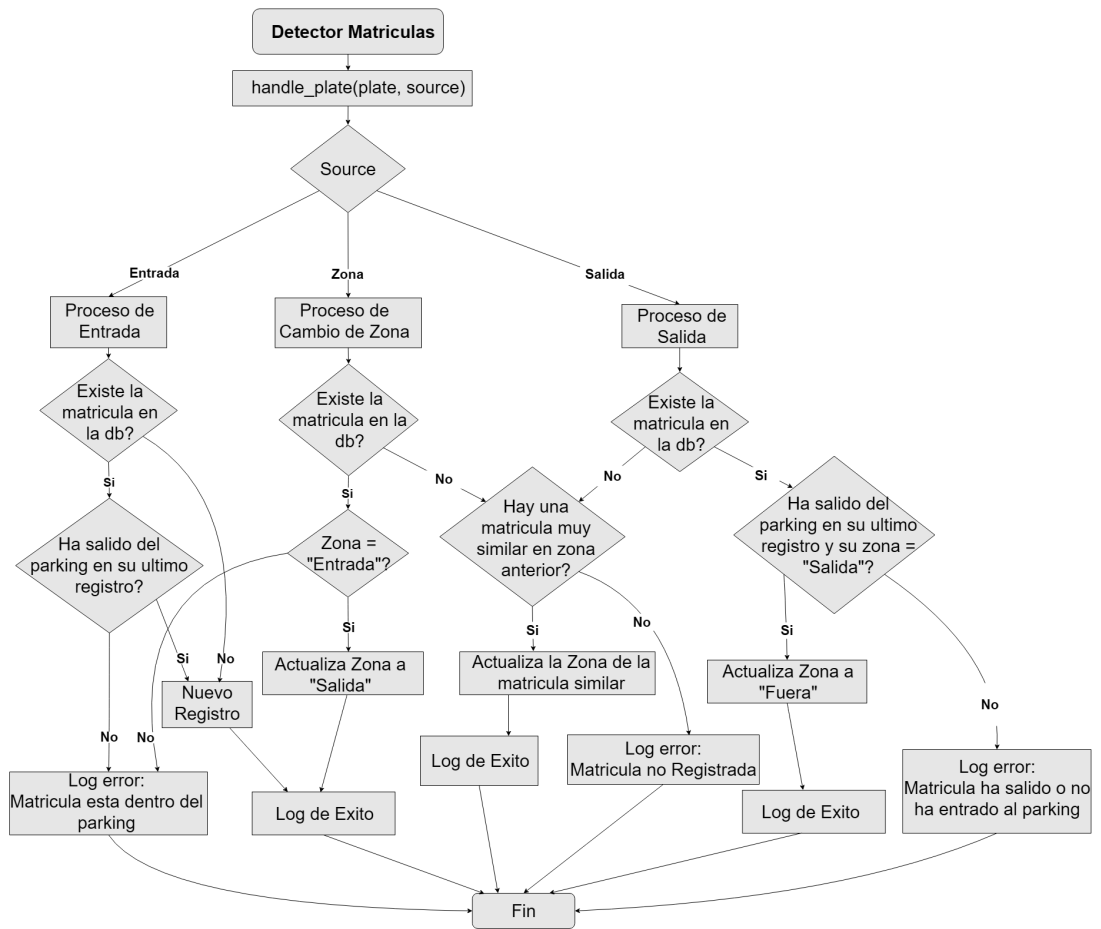


Figura 6.10: Diagrama de flujo insertar matrículas y logs

Enlace a su implementación ->: [Diagrama de flujo del proceso de detección](#)

La función `handle_plate` se ejecuta tras una detección válida de matrículas, es el núcleo del sistema de gestión de plazas, encargándose de procesar las matrículas detectadas(`plate`), según su cámara de origen, `source`(entrada, zona o salida), crear logs y actualizar la base de datos en consecuencia.

6.3.4.3.1. Funcionamiento general: El proceso de decisión se basa en obtener el último registro de la matrícula en la base de datos mediante `get_latest_plate_record(plate.text)`, y aplicar distintas acciones según su zona:

Entrada:

- Si ya salió previamente, se registra como nueva entrada.
- Si no tiene registros previos, se registra como nueva entrada.
- Si la matrícula ya existe y no ha salido, se detecta como duplicado y se genera un log de error.

Código 6.5: Archivo backend/services/db_service.py

```
1 if source == "Entrada":
2     # Si la matricula esta registrada
3     if latest_record:
4         # Si no ha salido
5         if latest_record["date_out"] is None:
6             # Log error
7
8         # Si ha salido, nueva entrada
9         else:
10            Plate.save_plate(plate)
11            # Log exito
12
13    # No hay registros, nueva entrada
14    else:
15        Plate.save_plate(plate)
16        # Log exito
```

Zona interior:

- Si el último estado fue "Zona 1", se actualiza a "Zona 2", y se crea un log de cambio de zona.
- Si no existe en la base de datos o si ya está en "Zona 2", o fuera, se genera un log de error.

Código 6.6: Archivo backend/services/db_service.py

```
1 elif source == "Zona":
2     # Si la matricula esta registrada
3     if latest_record:
4         # Si no ha salido y su ultimo registro es en la
          zona 1
5         if latest_record["date_out"] is None and
          latest_record["zona"] == "Zona 1":
6             update_plate_zona(latest_record["_id"], "Zona
          2")
7             # Log exito
8         else:
9             # Log error
10
11     # Si la matricula NO esta registrada
12     else:
13         # Si existe matricula casi igual en la zona 1
14         similar_plate = find_similar_plate(plate.
          license_plate_text, "Zona 2", plate.vehicle)
15         if similar_plate:
16             update_plate_date_out(similar_plate["_id"], "
          Zona 2")
17             # Log exito
18         else:
19             send_telegram_notis(plate.license_plate_text,
          plate.vehicle, source)
20             # Log error
```

Salida:

- Si el vehículo estaba en "Zona 2", y no ha salido, se registra su salida actualizando el campo date_out.
- Si no existe registro previo, o si ya ha salido o no estaba en "Zona 2", se genera un log de error.

Código 6.7: Archivo backend/services/db_service.py

```
1 elif source == "Salida":
2     # Si la matricula esta registrada
3     if latest_record:
4         # Si no ha salido y su ultimo registro es en la
          zona 2
5         if latest_record["date_out"] is None and
          latest_record["zona"] == "Zona 2":
6
7             update_plate_date_out(latest_record["_id"], "
          fuera")
```

```

8         # Log exito
9     else:
10         # Log error
11     # Si la matricula NO esta registrada
12     else:
13         # Si existe matricula casi igual en la zona 2
14         similar_plate = find_similar_plate(plate.
15             license_plate_text, "fuera", plate.vehicle)
16         if similar_plate:
17             update_plate_date_out(similar_plate["_id"], "
18                 Zona 2")
19             # Log exito
20         else:
21             send_telegram_notis(plate.license_plate_text,
22                 plate.vehicle, source)
23         # Log error

```

6.3.4.3.2. Funcionamiento en caso de errores de detección

Como el sistema no es 100% fiable, he implementado una función que ayuda a reducir errores en el reconocimiento de matrículas, `find_similar_plate`. Esta función recibe como parámetros la matrícula detectada (`plate_text`), la zona a la que quiere entrar (`zona`) y el tipo de vehículo (`vehicle`).

Esta función se activa cuando se detecta una matrícula dentro del parking que no ha pasado por la entrada. El algoritmo compara la matrícula detectada con las matrículas registradas en la zona anterior, evaluando cuántos caracteres coinciden en la misma posición.

Solo se considera una coincidencia válida si hay al menos **6 caracteres iguales y en la misma posición**. Si se encuentra una matrícula que cumple este criterio, se toma como la matrícula correcta y se actualiza su zona, evitando así que se quede registrada erróneamente en la zona anterior.

Código 6.8: Archivo backend/services/db_service.py

```

1 for plate in plates_in_previous_zone:
2     score = similarity_score(plate_text, plate["plate
3         "])
4
5     if score > best_score:
6         best_score = score
7         best_match = plate
8         break
9
10 return best_match

```

```

11 def similar_score(plate1, plate2):
12     same_chars = 0
13     not_same = 0
14
15     for i in range(len(plate1)):
16         if plate1[i] == plate2[i]:
17             same_chars += 1
18         else:
19             not_same += 1
20             if not_same == 2:
21                 break
22     return same_chars

```

Para una mayor seguridad en caso de detectar matrículas incorrectas se ha implementado el envío de mensajes mediante Telegram usando su API oficial con la función `send_telegram_notis()`, para notificar al administrador en tiempo real.

Creación del bot: He utilizado el bot oficial de Telegram@BotFather para crear un nuevo bot (@parking111_bot), desde el cual se obtiene el `BOT_TOKEN`.

Obtención del chat ID: Tras enviar un mensaje al bot (cualquier mensaje), se accede a:

https://api.telegram.org/bot<BOT_TOKEN>/getUpdates

El chat ID es la línea azul que aparece en el JSON de respuesta, como se muestra en la siguiente figura para diferentes dispositivos, que diferencia la línea roja.



Figura 6.11: Respuesta de la API de Telegram con el chat_id

Configuración: Se han guardado las credenciales en variables de entorno:

```
1 BOT_TOKEN = "8094717663:
    AAHNQJvBuyMuqLQNUeY1j3E_MPuRG8fFV5w"
2 CHAT_ID = "6493198654"
```

Envío de mensajes: Cuando se detecta una matrícula dentro del parking que no ha sido detectada en la entrada, ni se parece a ninguna de la zona anterior, se realiza una petición POST a la API de Telegram, esta notificación le llegaría al móvil del administrador del sistema, quien podrá tomar las medidas que el considere.

Código 6.9: Envío de mensaje desde el sistema

```
1 url = f"https://api.telegram.org/bot{bot_token}/
    sendMessage"
2 data = {
3     "chat_id": chat_id,
4     "text": message,
5     "parse_mode": "Markdown"
```

```

6 }
7 requests.post(url, data=data)

```

6.3.4.3.3. Funciones auxiliares:

- `get_latest_plate_record`: Consulta el registro más reciente de una matrícula.
- `update_plate_zona`: Cambia el estado del vehículo entre zonas.
- `update_plate_date_out`: Marca el vehículo como fuera del parking.
- `similar_score`: Calcula cuantos dígitos son iguales entre dos matrículas.

6.3.4.4. Servicio de Usuarios

El servicio de usuarios, implementado en `db_users.py`, permite gestionar el registro, la autenticación y los permisos dentro del sistema.

Código 6.10: Archivo `backend/services/db_users.py`

```

1 client = pymongo.MongoClient(MONGO_URI)
2 db = client[MONGO_DB]
3 collection = db['users']
4
5 def register_user(email, plain_password):
6     if not User.is_valid_email(email):
7         return "invalid_email"
8
9     if collection.find_one({"email": email}):
10        return "email_exists"
11
12    user = User.create(email, plain_password)
13
14    save_user_to_db(user)
15
16    return "success"
17
18 def verify_login(email, plain_password):
19    user_data = collection.find_one({"email": email})
20
21    if user_data:
22        user = User(user_data["email"], user_data["password"], user_data["admin"])
23        return user.check_password(plain_password)
24
25    return False

```

Enlace al Diagrama de Secuencia ->: [*Diagrama de secuencia de autenticación*](#)

6.4. Detección de Matrículas

6.4.1. Integración de Modelos de Visión por Computador

En el sistema de gestión de plazas de parking, se han integrado dos tecnologías de visión por computador para la detección y reconocimiento de matrículas:

- **Ultralytics (YOLOv8)**: Utilizado para la detección inicial de vehículos en los frames capturados por las cámaras IP. El modelo ha sido configurado para detectar específicamente coches (clase 2) y motos (clase 3).
- **Fast-ALPR**: Una vez detectado el vehículo, se emplea esta biblioteca para el reconocimiento de la matrícula mediante un pipeline especializado con detección y OCR.

6.4.2. Flujo de Detección

El flujo completo del proceso es el siguiente:

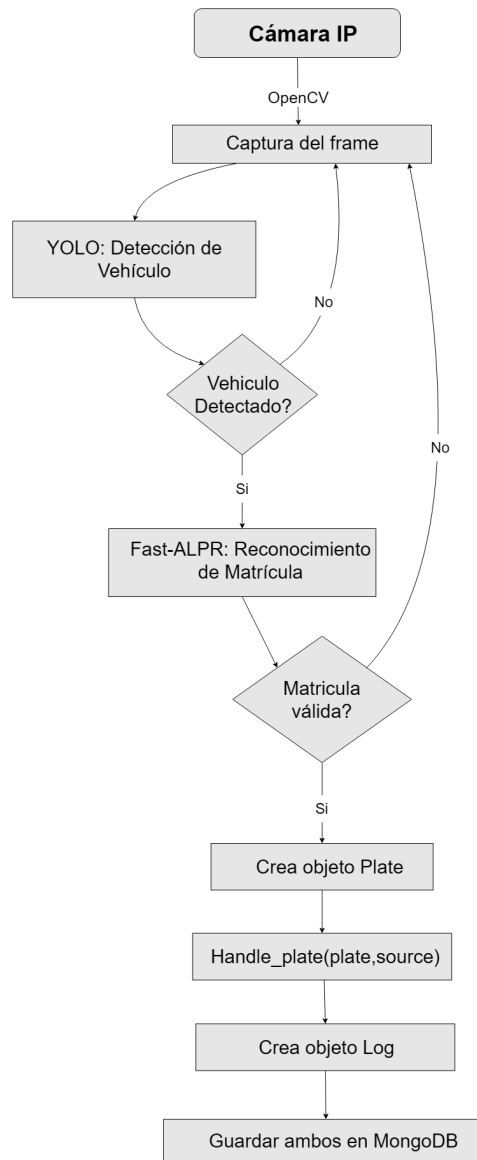


Figura 6.12: Diagrama de flujo de detección y reconocimiento de matrículas.

Enlace a implementación detectar Vehículo—>: [Implementación](#)

Enlace a implementación manejar Matrícula—>: [Implementación](#)

6.4.3. Ejemplo de Detección

Ejemplo de como sería la detección del vehículo usando YOLO(Recuadro azul) y detección de su matrícula usando Fast-ALPR(Recuadro verde) desde una cámara a la entrada del parking de la escuela:



Figura 6.13: Ejemplo sobre detección de vehículo y matrícula

Para que el sistema proceda a manejar la matrícula y el vehículo, necesita que la confianza de detección del vehículo sea superior a 0.85(85%) y que la matrícula sea superior a 0.95(95%).

El sistema incluye una función de validación de la matrícula que lee. El método `is_valid_plate` verifica si el formato de la matrícula es correcto, soportando tanto formatos modernos (4 dígitos + 3 letras) como los antiguos (1-2 letras + 4 dígitos + 1-2 letras) para matrículas españolas.

6.5. Base de Datos

El sistema de gestión de parking utiliza **MongoDB** como sistema de base de datos. En concreto, uso **MongoDB Atlas**, la plataforma de base de datos como servicio (DBaaS) de MongoDB que permite gestionar los datos de forma remota y segura a través de la nube. Esta elección facilita la escalabilidad y simplifica la gestión del almacenamiento sin necesidad de desplegar infraestructura local adicional [14].

6.5.1. Estructura General

La base de datos MongoDB se organiza en colecciones, y su acceso desde el backend se gestiona gracias a la biblioteca PyMongo. La conexión se establece utilizando la URI y el nombre de la base de datos definidos en un archivo de configuración, en la carpeta config, en el archivo config.py.

Cada módulo del backend que necesita entrar a la base de datos inicializa su propia conexión utilizando estas configuraciones.

Código 6.11: Conexión a MongoDB

```
1 from pymongo import MongoClient
2 from config.back_config import MONGO_URI, PARKING
3
4 client = MongoClient(MONGO_URI)
5 db = client[MONGO_PARKING]
6 collection = db["vehicles"]
```

La base de datos está compuesta por tres colecciones principales:

- **vehicles**: Registra información sobre cada vehículo detectado en el parking (entrada, salida, zona, etc.).
- **logs**: Almacena un historial detallado de acciones y eventos del sistema (entradas, salidas, errores, etc.).
- **users**: Gestiona la autenticación y los roles de los usuarios registrados.

6.5.2. Inserción de Datos

La inserción de documentos en MongoDB se realiza mediante los modelos definidos en el backend. El método para introducir matrículas y su log correspondiente al entrar al parking es:

Código 6.12: Archivo backend/models/plate.py

```
1 client = MongoClient(MONGO_URI)
2 db = client[MONGO_PARKING]
3 collection = db['vehicles']
4
5 def save_plate(cls, plate):
6     plate_data = {
7         "plate": plate.license_plate_text,
8         "confidence": plate.confidence,
9         "vehicle": plate.vehicle,
10        "date_in": plate.date_in,
11        "date_out": plate.date_out,
12        "zona": plate.zona
13    }
14
15    collection.insert_one(plate_data)
16
17    log = Log(
18        action="Entrada",
19        description=f"{plate.vehicle} con matricula {plate.
20            license_plate_text} entro al parking a
21            las {plate.date_in}",
22        plate=plate.license_plate_text,
23        zona=plate.zona,
24        timestamp=datetime.now().strftime("%Y-%m-%d %
25            H:%M:%S")
26    )
27
28    Log.save_log(log)
29
30    return plate
```

6.5.3. Consulta de Datos

El sistema expone varias rutas API a través de Flask para consultar la base de datos, la ruta para consultar las plazas disponibles de cada zona del parking es:

Código 6.13: Archivo backend/api/plazas.py

```
1 client = MongoClient(MONGO_URI)
2 db = client[MONGO_PARKING]
3 collection = db['vehicles']
4
5 parking_bp = Blueprint("parking", __name__)
6
7 @parking_bp.route("/api/spots", methods=["GET"])
8 def parking_status():
9     occupied_entry_car = collection.count_documents({"
10         zona": "entrada", "vehicle": "coche"})
11     occupied_exit_car = collection.count_documents({"zona
12         ": "salida", "vehicle": "coche"})
13
14     occupied_entry_moto = collection.count_documents({"
15         zona": "entrada", "vehicle": "moto"})
16     occupied_exit_moto = collection.count_documents({"
17         zona": "salida", "vehicle": "moto"})
18
19     available_entry_spots_car = max(0,
20         TOTAL_ENTRY_SPOTS_CAR - occupied_entry_car)
21     available_exit_spots_car = max(0,
22         TOTAL_EXIT_SPOTS_CAR - occupied_exit_car)
23
24     available_entry_spots_moto = max(0,
25         TOTAL_ENTRY_SPOTS_MOTO - occupied_entry_moto)
26     available_exit_spots_moto = max(0,
27         TOTAL_EXIT_SPOTS_MOTO - occupied_exit_moto)
28
29     return jsonify({
30         "entrada_coche": available_entry_spots_car,
31         "salida_coche": available_exit_spots_car,
32         "entrada_moto": available_entry_spots_moto,
33         "salida_moto": available_exit_spots_moto
34     })
```

6.6. Frontend

El frontend del sistema de gestión de parking está construido con Flet, un framework de Python que permite crear interfaces de usuario web y de escritorio utilizando Flutter. La estructura del frontend sigue un patrón de vistas y componentes reutilizables.

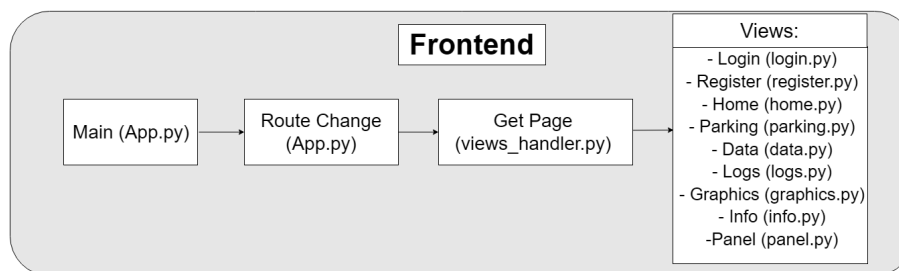


Figura 6.14: Frontend

6.6.1. Inicialización del Sistema

El archivo app.py es el encargado de iniciar la ejecución de la interfaz gráfica del usuario. Desde aquí se configura el entorno, se define la navegación entre vistas y se lanza la interfaz que permitirá la interacción del usuario con el sistema.

Código 6.14: Archivo frontend/app.py

```
1 def main(page: ft.Page):
2     def route_change(route):
3         page.views.clear()
4
5         view = get_page(page)
6
7         if view:
8             page.views.append(view)
9         else:
10            page.session.clear()
11            page.go("/login") # ruta desconocida
12            page.update()
13
14        page.theme_mode="light"
15        page.adaptive=True
16        page.on_route_change = route_change
17        page.go("/login") # Ruta inicial
18
19 #, view=ft.WEB_BROWSER
20 ft.app(target=main)
```

6.6.2. Vistas

6.6.2.1. Vista Login (login.py)

La vista de login es el punto de entrada para los usuarios. Permite la autenticación mediante correo electrónico y contraseña, verificando las credenciales en la base de datos. Si las credenciales son correctas, los datos del usuario se almacenan en la sesión y se redirige a la página principal.

Código 6.15: frontend/views/login.py:4-26

```
1 def login(page):
2     def on_login_click(e):
3         email = gmail_field.value
4         password = pass_field.value
5
6         if email and password:
7             if verify_login(email, password):
8                 user_data = {
9                     "email": email,
10                    "is_admin": is_admin(email),
11                }
12                page.session.set("user", user_data)
13                page.client_storage.set("user", user_data)
14                page.go("/home")
15            else:
16                lbl_error.value = "Usuario o contraseña
17                                incorrectos"
18        else:
19            lbl_error.value = "Por favor, ingrese usuario
20                            y contraseña"
21
22        lbl_error.update()
23        page.update()
```

Enlace al Diseño->: [Diseño de Login](#)

Enlace al Diagrama de Secuencia->: [Diagrama de Secuencia](#)

6.6.2.2. Vista Registro (register.py)

Esta vista permite a los usuarios crear cuentas nuevas en el sistema. Realiza validaciones para asegurar que las contraseñas coincidan y que el correo electrónico no esté registrado previamente, mostrando mensajes de error cuando falla. Al registrarse correctamente, el usuario es redirigido automáticamente a la página principal.

Código 6.16: frontend/views/register.py:5-35

```
1 def register(page):
2     def on_register_submit(e):
3         email = register_email_field.value
4         password = register_pass_field.value
5         confirm_password = confirm_register_pass_field.value
6
7         if password != confirm_password:
8             lbl_error.value = "Las contraseñas no coinciden"
9         elif email and password:
10            result = register_user(email, password)
11            if result == "success":
12                user_data = {
13                    "email": email,
14                    "is_admin": False,
15                }
16                page.session.set("user", user_data)
17                page.client_storage.set("user", user_data)
18                page.go("/home")
19            elif result == "email_exists":
20                lbl_error.value = "El correo electrónico ya está registrado"
21            elif result == "invalid_email":
22                lbl_error.value = "El formato del correo electrónico es inválido"
23        else:
24            lbl_error.value = "Por favor, ingrese los datos correctamente"
25
26        lbl_error.update()
27        page.update()
```

Enlace al Diseño—>: [Diseño de Registro](#)

Enlace al Diagrama de Secuencia—>: [Diagrama de Secuencia](#)

6.6.2.3. Vista Tarjetas (frontv2.py)

La vista principal muestra el estado actual del parking en tiempo real mediante tarjetas que indican la disponibilidad de plazas para coches y motos en cada zona. Se utilizan solicitudes asíncronas periódicas para mantener los datos actualizados de forma automática cada cinco segundos.

Código 6.17: frontend/views/frontv2.py:85–98

```
1  async def update_parking_status(self, page):
2      while True:
3          if page.route != "/home":
4              break
5          try:
6              async with ClientSession() as session:
7                  async with session.get(API_URL_SPOTS) as
                        response:
8                      if response.status == 200:
9                          data = await response.json()
10                         self.zone_a.update_status(data.get("entrada_coche"), data.get("entrada_moto"))
11                         self.zone_b.update_status(data.get("salida_coche"), data.get("salida_moto"))
12
13         except Exception as e:
14             print("Error al obtener datos de la API:", e)
15             await asyncio.sleep(5)
```

Enlace al Diseño→:[Diseño de Tarjetas](#)

6.6.2.4. Vista Parking (parking.py)

Esta vista ofrece una representación visual alternativa que simula la distribución física del parking construida con containers de flet. Muestra en tiempo real las plazas disponibles para coches y motos en sus diferentes zonas, y se actualiza en tiempo real mediante llamadas asíncronas a la API.

Código 6.18: frontend/views/parking.py:58–78

```
1  # Contenedores principales
2      self.zona_entrada = ft.Container(
3          content=ft.Container(
4              content=ft.Column([
5                  ft.Row([Entrada], alignment=ft.
6                      MainAxisAlignment.CENTER),
7                  ft.Row([Car_icon, self.
8                      plazas_zona_entrada_coche],
9                      alignment=ft.MainAxisAlignment.
10                         CENTER),
11                  ft.Row([Moto_icon, self.
12                      plazas_zona_entrada_moto],
13                      alignment=ft.MainAxisAlignment.
14                         CENTER),
15              ]),
16              width=130 * self.scale,
17              height=220 * self.scale,
18              padding=20,
19              bgcolor=ft.colors.BLUE,
20              border_radius=ft.border_radius.only(
21                  bottom_right=10, bottom_left=10),
22          ),
23          margin=ft.margin.only(top=130 * self.scale)
24      )
25
26      self.curva = ft.Container(
27          content=ft.Container(
28              width=130 * self.scale,
29              height=130 * self.scale,
30              bgcolor=ft.colors.TEAL_500,
31              border_radius=ft.border_radius.only(
32                  top_left=100)
33          ),
34          margin=ft.margin.only(left=0, top=0)
35      )
```

Enlace al Diseño->: [Diseño alternativo Estructura](#)

6.6.2.5. Vista Panel de Administrador (panel.py)

Esta vista ofrece una interfaz completa para la gestión de matrículas en el parking, dividida en dos secciones, registro y eliminación. Permite al administrador añadir nuevas matrículas y eliminarlas individualmente o en conjunto.

Destaca la funcionalidad de cálculo del tiempo y que colorea las horas según su duración (verde para menos de 8 horas, naranja para 8-24 horas, y rojo para más de 24 horas)

Código 6.19: frontend/views/panel.py:131-162

```
1  for r in registros:
2      plate = r.get("plate")
3      zona = r.get("zona")
4      tipo = r.get("vehicle")
5      date_in = r.get("date_in")
6
7      matricula=Plate(plate, r.get("confidence"), tipo,
8                      date_in, zona)
9
10     date_in_str = datetime.fromisoformat(r["date_in"]
11                                         ])
12     diff = now - date_in_str
13
14     horas = diff.total_seconds() / 3600
15     horas_tabla = f"{horas:.2f} h"
16
17     color_horas = ft.colors.GREEN
18     if horas > 24:
19         color_horas = ft.colors.RED
20     elif horas > 8:
21         color_horas = ft.colors.ORANGE
22
23     tabla.rows.append(ft.DataRow(cells=[
24         ft.DataCell(ft.Text(plate, color=ft.colors.
25                           WHITE)),
26         ft.DataCell(ft.Text(zona, color=ft.colors.
27                           WHITE)),
28         ft.DataCell(ft.Text(tipo, color=ft.colors.
29                           WHITE)),
30         ft.DataCell(ft.Text(date_in, color=ft.colors.
31                           WHITE)),
32         ft.DataCell(ft.Text(horas_tabla, color=
33                           color_horas)),
34         ft.DataCell(ft.IconButton(
35             icon=ft.icons.DELETE,
36             icon_color=ft.colors.RED_400,
37             on_click=lambda e, p=matricula:
38                 delete_one_plate(e, p)
```

```

31         ))
32     )))

```

Enlace al Diseño—>: [Diseño Panel de Control](#)

6.6.2.6. Vista Data (data.py)

Muestra información histórica de los vehículos que han accedido al parking, incluyendo matrículas, zonas y fechas de entrada y salida. La vista permite realizar búsquedas por matrícula, navegar entre páginas de resultados mediante paginación y actualizar los datos con un botón.

Código 6.20: frontend/views/data.py:127–154

```

1  # Layout principal
2      logs_layout = ft.Column(
3          [
4              ft.Text("Data", size=24, weight=ft.FontWeight
5                  .BOLD),
6              ft.Row(
7                  [table],
8                  scroll=ft.ScrollMode.AUTO,
9                  vertical_alignment=ft.CrossAxisAlignment.
10                     START
11                 ),
12             ft.Row([search_field, btn_search], alignment=
13                 ft.MainAxisAlignment.CENTER, spacing=15),
14             ft.Row([btn_prev, btn_refresh, btn_next],
15                 alignment=ft.MainAxisAlignment.CENTER,
16                 spacing=20),
17             page_counter,
18         ],
19         alignment=ft.MainAxisAlignment.CENTER,
20         horizontal_alignment=ft.CrossAxisAlignment.CENTER
21     )
22
23     page.appbar = NavBar(page)
24     # Datos primera vez
25     update_data()
26
27     return ft.View(
28         "/data",
29         [logs_layout],
30         appbar=page.appbar,
31         horizontal_alignment=ft.CrossAxisAlignment.CENTER,
32         scroll=ft.ScrollMode.AUTO
33     )

```

Enlace al Diseño—>: [Diseño de Datos](#)

6.6.2.7. Vista Logs (logs.py)

Esta vista muestra información histórica sobre los logs del sistema. Tiene una tabla con registros que incluyen acción, descripción, matrícula, zona y su timestamp. Permite paginar los resultados y actualizar los datos manualmente con un botón.

Código 6.21: frontend/views/logs.py:87–113

```
1 # Funcion datos API
2 def update_data():
3     nonlocal current_page, total_pages
4     params = {
5         "page": current_page,
6         "limit": limit,
7     }
8
9     try:
10         response = requests.get(API_URL_LOGS, params=
11                                 params)
12         if response.status_code == 200:
13             data = response.json()
14             registros = data["data"]
15             total = data["total"]
16             total_pages = (total // limit) + (1 if
17                             total % limit > 0 else 0)
18             update_rows(registros)
19             page_counter.value = f"Pagina {
20                                 current_page} de {total_pages}"
21             page.update()
22         except requests.RequestException as e:
23             print(f"Error: {e}")
```

Enlace al Diseño->: [Diseño de Logs](#)

6.6.2.8. Vista Info (info.py)

Vista pensada para mostrarse en una pantalla situada en la entrada del parking. Presenta en tiempo real la disponibilidad de plazas por zonas y tipo de vehículo (coches y motos), actualizándose automáticamente mediante peticiones periódicas a la API. También muestra la fecha y hora actual, y adapta su diseño en función de la orientación de la pantalla.

Código 6.22: frontend/views/info.py:97-114

```
1  async def update_status(self, page: ft.Page):
2      while page.route == "/info":
3          try:
4              async with ClientSession() as session:
5                  async with session.get(API_URL_SPOTS) as
6                      response:
7                      if response.status == 200:
8                          data = await response.json()
9
10                     self.update_input(self.z1_car, "
11                         coches:", data.get('
12                             entrada_coche', 0))
13                     self.update_input(self.z1_moto, "
14                         motos:", data.get('
15                             entrada_moto', 0))
16                     self.update_input(self.z2_car, "
17                         coches:", data.get('
18                             salida_coche', 0))
19                     self.update_input(self.z2_moto, "
20                         motos:", data.get('
21                             salida_moto', 0))
22
23                     self.update_date_time()
24                     self.page.update()
25             except Exception as e:
26                 print("Error al obtener datos de la API:", e)
27                 await asyncio.sleep(5)
```

Enlace al Diseño—>: [Diseño fijo Parking](#)

6.6.2.9. Archivo Gestor de Vistas (views_handler.py)

Todas las vistas anteriores son gestionadas mediante un sistema de enrutamiento que dirige al usuario a la vista correspondiente según la URL solicitada, haciendo sus previas comprobaciones.

Código 6.23: frontend/views_handler.py:11-43

```
1 def get_page(page: ft.Page):
2     route = page.route
3     user = page.session.get("user")
4     if not user and route not in ("/login", "/register"):
5         return None
6
7     if user and route in ("/login", "/register"):
8         page.session.clear()
9
10    if user and route in ("/data", "/info", "/logs", "/
11    graphics") and not user.get("is_admin"):
12        return parking_page(page)
13
14    match route:
15        case "/login":
16            return login(page)
17        case "/register":
18            return register(page)
19        case "/home":
20            return parking_page(page)
21        case "/parking":
22            return parking(page)
23        case "/data":
24            return data(page)
25        case "/logs":
26            return logs(page)
27        case "/info":
28            return info_page(page)
29        case "/graphics":
30            return graphics_page(page)
31
32    case _:
33        return None
```

6.7. Componentes Frontend

6.7.1. Gráficas

Esta vista ofrece visualizaciones estadísticas sobre el uso del parking, incluyendo distribución de tipos de vehículos, ocupación por zonas y entradas por día al parking. Están organizadas en filas.

Código 6.24: frontend/views/graphics.py:42–63

```
1 def graphics_page(page: ft.Page):  
2     page.appbar = NavBar(page)  
3  
4     entradas_dia, ocupacion_zonas, tipos_vehiculos =  
5         obtener_datos()  
6  
7     layout = ft.Column([  
8         ft.Row([Graf1(tipos_vehiculos), Graf2(  
9             ocupacion_zonas)], expand=True),  
10        ft.Row([Graf3(entradas_dia)], expand=True)  
11    ], spacing=20, expand=True)
```

Enlace al Diseño—>: [Diseño de las Gráficas](#)

Cada gráfica la he encapsulado en una clase que hereda de `ft.Container`. Cada clase recibe los datos ya procesados y construye dinámicamente los elementos visuales correspondientes.

En el caso de Graf1, se utiliza un componente PieChart para representar la proporción de tipos de vehículos registrados. Para cada sección uso un componente PieChartSection, el gráfico se genera a partir de los datos con los que se crea la clase y se le asigna un color según el tipo (*azul* para coches, *verde* para motos), además de incluir un icono para cada tipo.

Código 6.25: frontend/components/graphics.py:10–36

```
1 class Graf1(ft.Container):
2     def __init__(self, tipos_vehiculos):
3         super().__init__(**same_style)
4         self.normal_radius = 110
5         self.normal_badge_size = 50
6         total_vehiculos = sum(tipos_vehiculos.values())
7         title = ft.Text("Tipos de Vehiculos", size=20,
8             weight=ft.FontWeight.BOLD, color=ft.Colors.
9             BLACK)
10
11         chart = ft.PieChart(
12             sections=[
13                 ft.PieChartSection(
14                     count,
15                     title=f"{{(count / total_vehiculos) *
16                         100:.1f}}%",
17                     color="blue" if tipo == "coche" else
18                         "green",
19                     radius=self.normal_radius,
20                     badge=ft.Icon(
21                         ft.Icons.DIRECTIONS_CAR if tipo
22                         == "coche" else ft.Icons.
23                         TWO_WHEELER,
24                         color="#84c3e3",
25                         size=self.normal_badge_size
26                     ),
27                     badge_position=1
28                 )
29                 for tipo, count in tipos_vehiculos.items
30                 ()
31             ],
32             sections_space=0,
33             center_space_radius=0,
34         )
```

La clase Graf2 implementa una gráfica de barras usando BarChart, donde cada grupo de barras representa una zona del parking y la altura indica el número de vehículos estacionados. Las barras utilizan un gradiente de color cian y están acompañadas de etiquetas en los ejes. El eje X muestra los nombres de las zonas, mientras que el eje Y se ajusta automáticamente según el número máximo de vehículos de alguna zona.

Código 6.26: frontend/components/graphics.py:93–115

```
1 chart = ft.BarChart(  
2     bar_groups=bar_groups,  
3     border=ft.border.all(1, ft.Colors.BLACK),  
4     bottom_axis=ft.ChartAxis(labels=x_labels,  
5         labels_size=40),  
6     left_axis=ft.ChartAxis(  
7         title=ft.Container(ft.Text("Numero de  
8             Vehiculos", color=ft.Colors.BLACK)),  
9         title_size=40,  
10        labels=y_labels,  
11    ),  
12    horizontal_grid_lines=ft.ChartGridLines(  
13        color=ft.Colors.with_opacity(0.3, ft.  
14            Colors.BLACK), width=1, dash_pattern  
15        =[3, 3]  
16    ),  
17    groups_space=20,  
18    max_y=y_max,  
19 )
```


Por último, la clase Graf3 construye un gráfico de líneas con LineChart, que muestra el número de entradas por día. Los datos se agrupan con la clase Counter de collections, se ordenan cronológicamente y se representan como puntos conectados por una línea curva de color cian. Se definen etiquetas tanto en el eje X (fechas) como en el eje Y (número de entradas). La escala del eje Y también se adapta automáticamente al valor máximo de los datos.

Código 6.27: frontend/components/graphics.py:144–185

```

1 line_chart = ft.LineChart(
2     data_series=[
3         ft.LineChartData(
4             data_points=data_points,
5             stroke_width=5,
6             curved=True,
7             color=ft.Colors.CYAN
8         )
9     ],
10    left_axis=ft.ChartAxis(
11        labels=y_labels,
12        labels_interval=5,
13        title=ft.Text("N de Entradas", color=ft.
14            Colors.BLACK),
15        title_size=40
16    ),
17    bottom_axis=ft.ChartAxis(
18        labels=x_labels,
19        labels_interval=1,
20        title=ft.Container(
21            ft.Text("Fecha", color=ft.Colors.
22                BLACK)
23        ),
24        title_size=20
25    ),
26    border=ft.border.all(1, ft.Colors.BLACK),
27    horizontal_grid_lines=ft.ChartGridLines(
28        color=ft.Colors.with_opacity(0.3, ft.
29            Colors.BLACK), width=1, dash_pattern
30            =[3, 3],
31    ),
32    vertical_grid_lines=ft.ChartGridLines(
33        color=ft.Colors.with_opacity(0.3, ft.
34            Colors.BLACK), width=1, dash_pattern
35            =[3, 3],
36    ),
37    max_y=y_max,
38    min_y=0,
39    expand=True,
40 )

```

6.7.2. Barra de Navegación

La barra de navegación está implementada como la clase `NavBar`, que extiende de `AppBar`. Su diseño está pensado para adaptarse automáticamente a diferentes tamaños de pantalla.

La estructura de la barra de navegación incluye el nombre de la Escuela ya que esta aplicación desde un principio está enfocada en el parking de la escuela, *Parking ETSIIT UGR* en su versión completa y *UGR* en pantallas pequeñas.

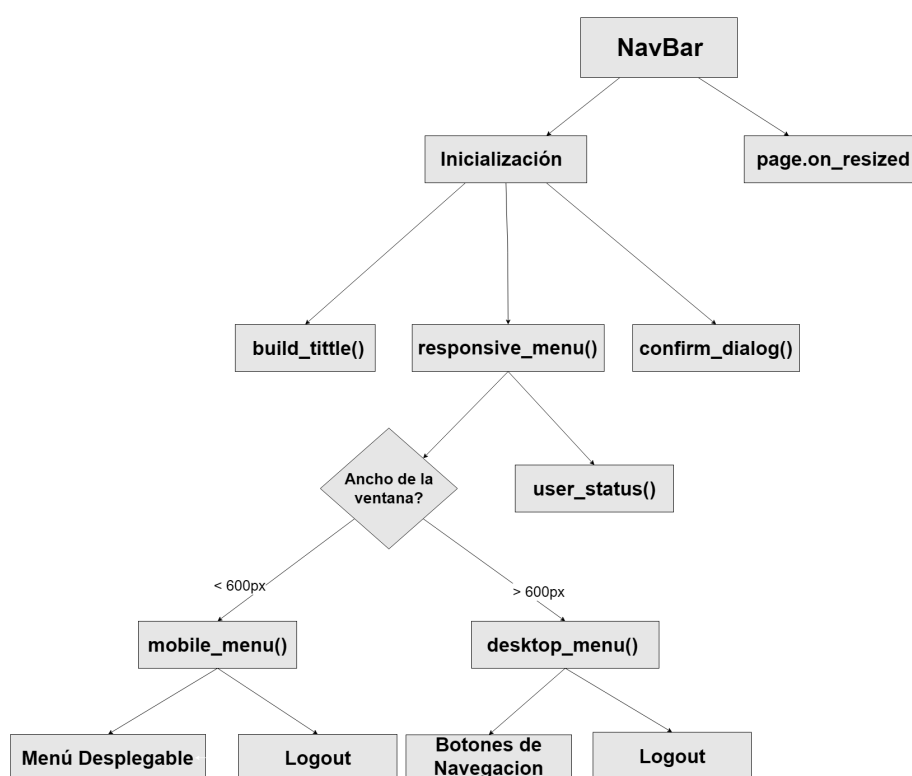


Figura 6.15: Esquema Barra de Navegación

Enlace al Diseño—>: [Diseño Barra de Navegación](#)

En pantallas de más de 600 px, se muestran todos los botones de navegación en la misma línea. Sin embargo, cuando el tamaño de la pantalla es menor, estas opciones se agrupan en un menú desplegable. Esto lo controla el evento `on_resized`, que detecta automáticamente los cambios de tamaño de ventana.

El contenido del menú cambia en base del rol del usuario. En el caso de los administradores, se incluyen accesos directos a todas las vistas del sistema, los usuarios normales, solo tienen acceso a las vistas que consultan las plazas disponibles.

La barra incorpora otras funcionalidades relevantes, una de ellas es el indicador que muestra el nombre del usuario actualmente logeado. También tiene una opción para cambiar entre el tema del sistema, claro u oscuro, y por último, tiene un botón para cerrar sesión que activa un diálogo de confirmación antes de desloguearse del sistema.

La barra de navegación se instancia en cada vista de la aplicación y cuando un usuario selecciona alguna opción, se invoca el método `page.go()` para redirigir la vista.

6.8. Autenticación de Usuarios

Desde el frontend, el usuario puede acceder a las vistas de login y registro, que permiten iniciar sesión o crear una nueva cuenta. Estas vistas están conectadas con el backend, donde se gestionan los datos de autenticación. La base de datos MongoDB se encarga de almacenar la información de los usuarios, incluyendo sus correos electrónicos y contraseñas protegidas mediante hashing.

El modelo user guarda información, como el correo que es el identificador único, la contraseña que se guarda como hash y un boolean que indica si es administrador. La generación y comparación de hashes se hace usando la biblioteca bcrypt.

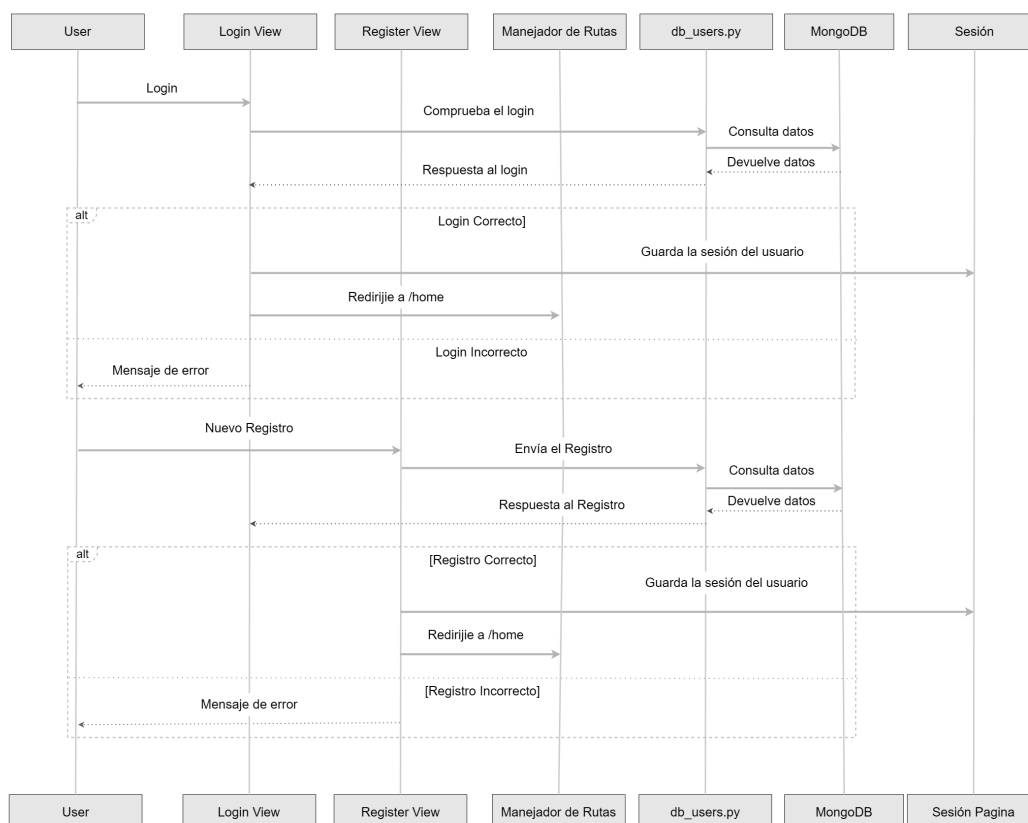


Figura 6.16: Diagrama de Secuencia Autenticación

Enlace al Diseño—>: [Vistas de Login y Register](#)

Enlace a las implementaciones de las funciones en db_users.py: [Implementación](#)

Para registrar un nuevo usuario, la aplicación valida previamente que el correo electrónico no esté ya registrado. Si no lo está y tiene un formato válido, se crea un nuevo objeto usuario con la contraseña hasheada y se guarda en mongo.

Para el login, la verificación consiste en localizar al usuario en la base de datos con su correo, crear una instancia de su modelo con sus datos y comprobar que la contraseña que ha introducido coincide con el hash almacenado. Además, se puede comprobar si un usuario tiene permisos administrativos a través de la función `is_admin(email)`.

El logout está en un botón en la barra de navegación que, al pulsarlo, borra toda la información del usuario en la sesión actual. Una vez borrada la sesión, el sistema lo redirige a la vista de login.

En cuanto a la seguridad, las contraseñas se manejan mediante hashing con `bcrypt`, evitando el almacenamiento en texto plano. Además, se valida el formato del correo electrónico y la coincidencia de contraseñas durante el proceso de registro. También se verifica en el archivo `views_handler.py` que usuario puede acceder a cada ruta (además de los botones de la navbar que no se muestran).

7 | Pruebas y Validación

En esta sección se explica cómo el sistema se adapta a diferentes tamaños de pantalla y dispositivos. También se incluyen pruebas de adaptabilidad de la interfaz (responsive design). Además, se presenta una recopilación de grabaciones realizadas en diferentes sistemas operativos, donde se muestra el sistema, así como simulaciones del funcionamiento de la entrada y salida de vehículos en el parking.

7.1. Contenido Responsive

Para garantizar que el sistema se adapte a diferentes dispositivos y pantallas, he usado diferentes técnicas de contenido responsive que adaptan dinámicamente la interfaz según el tamaño de la pantalla.

Uno de los principales componentes que incorpora esta lógica es la barra de navegación (NavBar), la cual modifica su estructura a través del método `responsive_menu()`. Este método detecta automáticamente el ancho de la ventana mediante `page.window.width` y genera una interfaz distinta para móviles o escritorios:

- **Versión móvil (pantallas menores a 600px):** Muestra un menú desplegable usando `PopupMenuButton`, agrupando todas las opciones de navegación bajo un único botón.
- **Versión escritorio (pantallas de 600px o más):** Muestra los iconos individuales para cada opción de navegación.

En las siguientes fotos se muestra tanto la barra de navegación como las diferencias entre una pantalla grande y una pequeña.

Vista de tarjetas:

En la vista que muestra las plazas en forma de tarjetas, he utilizado el componente `ResponsiveRow` de Flet. Este componente permite organizar los elementos en columnas como si fuera un grid de css, que se adaptan automáticamente al tamaño de pantalla.

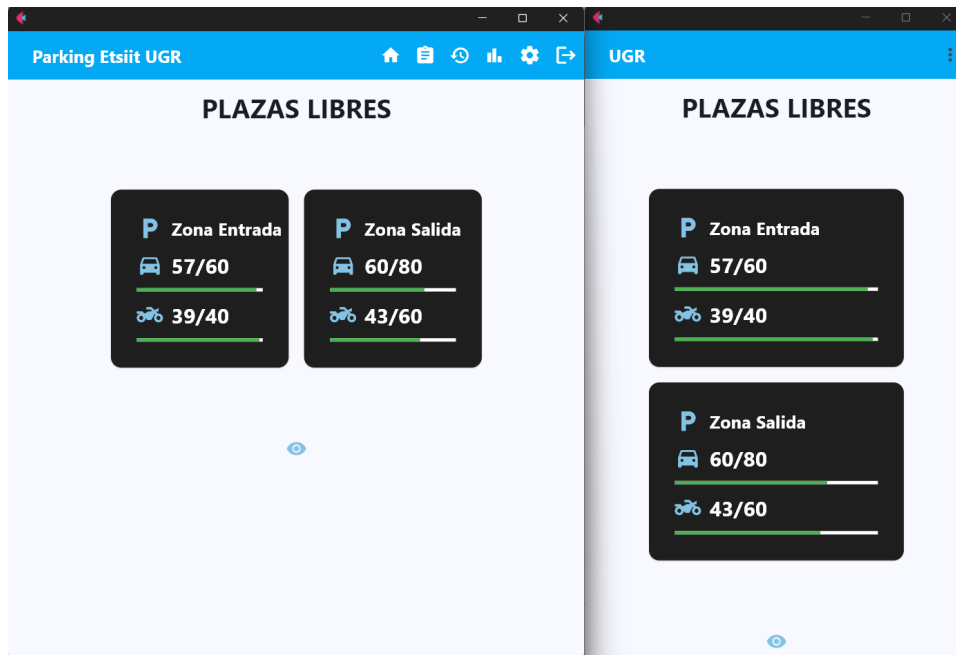


Figura 7.1: Tarjetas Responsive

El parámetro `col={"xs": 12, "sm": 5}` indica que en pantallas muy pequeñas (xs) el elemento ocupa 12 columnas (ancho completo), y a partir de pantallas pequeñas (sm), ocupa 5 columnas (aproximadamente la mitad) que se expanden según el tamaño de la pantalla.

Vista Estructura:

En la vista alternativa de la estructura del parking, utilizo un mecanismo propio de escalado dinámico. Calculo un factor de escala en función del ancho de la ventana con el método `on_resize()`, y ese factor de escala lo multiplico por el ancho y alto de los contenedores, haciendo que el parking se vea más grande para pantallas grandes o más pequeño para móviles.



Figura 7.2: Estructura Responsive

Vista Panel de Control:

El diseño responsive implementado en esta vista hace que, al detectar el ancho de la pantalla con `page.window.width`, si es menor a 600 píxeles, se establece `limit = 5`. En caso contrario, `limit = 7`. Limit determina cuántos registros se muestran en la tabla donde se eliminan matrículas: 5 en pantallas pequeñas y 7 en pantallas grandes.

Además, cuando `limit` es 5 (pantallas pequeñas), la fila (Row) que contiene los botones del formulario se reorganiza de forma vertical (Column), adaptándose mejor al espacio reducido.

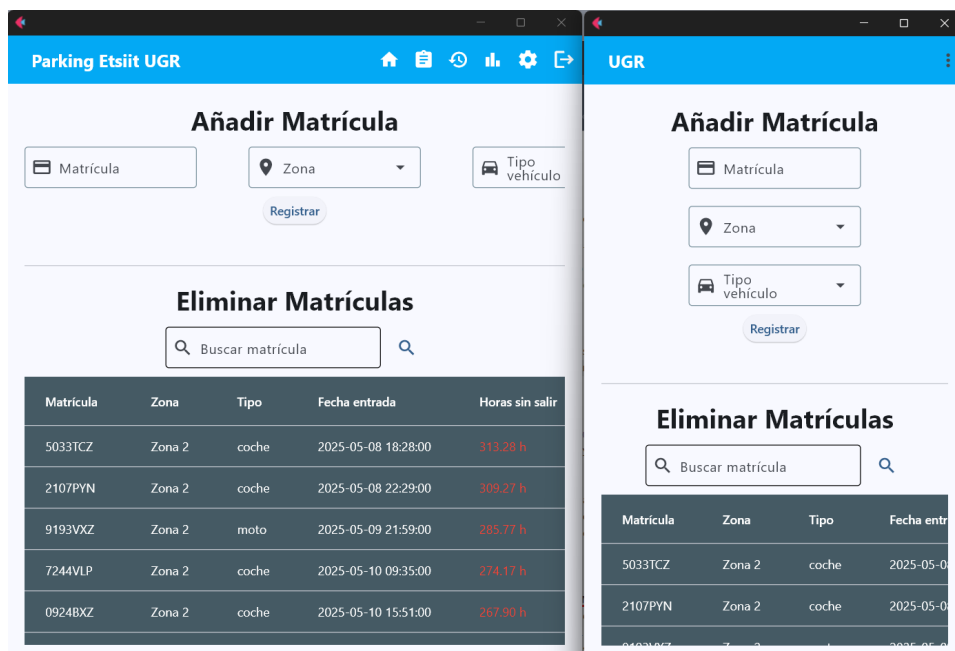


Figura 7.3: Panel Responsive

Vista Gráficas:

La vista de gráficas se adapta igualmente al 100% del tamaño de la pantalla, esto se consigue con la propiedad de `flet expand = True`, lo que permite que los gráficos aprovechen el máximo del espacio disponible.

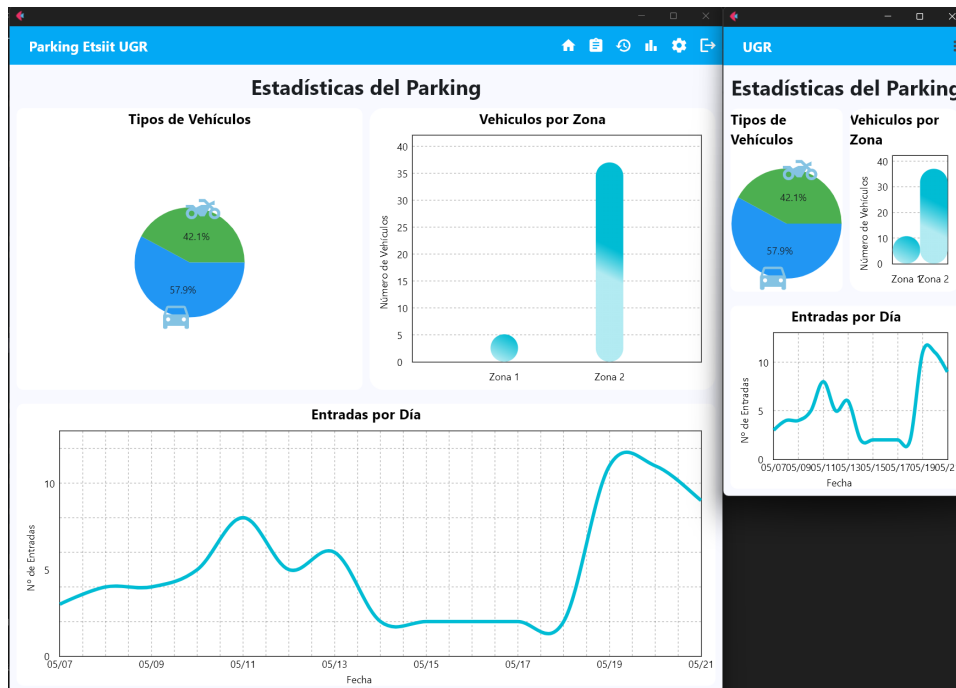


Figura 7.4: Gráficas Responsive

Vista Info:

Por último, en la vista fija de info del parking, se ha programado para que el texto reduzca/aumente automáticamente su tamaño según el ancho de la pantalla. Además, la alineación de los datos cambia dinámicamente según el mayor ancho o alto de la pantalla, esto es especialmente útil por si se decide poner una pantalla vertical.

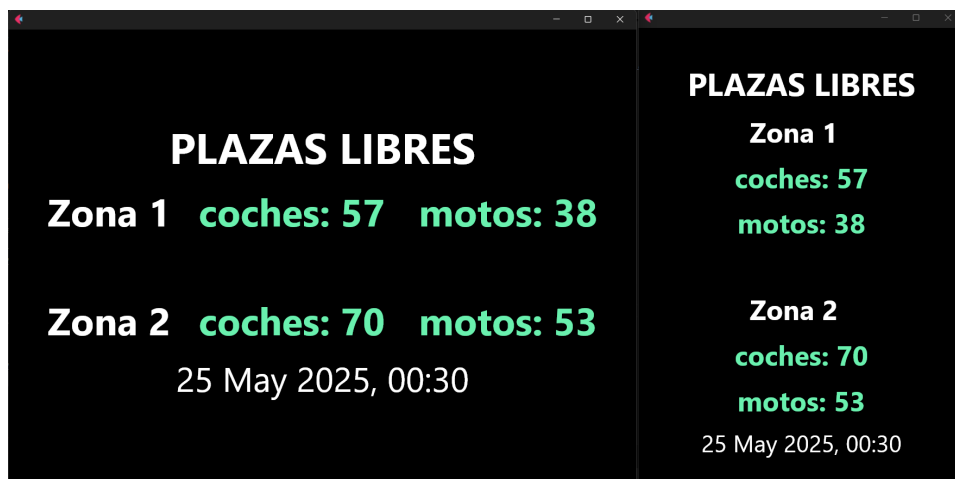


Figura 7.5: Info Responsive

7.2. Grabaciones del Resultado Final

Con el objetivo de comprobar el funcionamiento del sistema en su versión final, he hecho estas grabaciones que demuestran el sistema completo, así como una simulación que enseña cómo responde el sistema a la entrada y salida de vehículos.

- [Windows](#)
- [Linux](#)
- [Android](#)
- [iOS](#)
- [Tablet](#)
- [Navegador Web](#)
- [Simulación de entrada/salida de vehiculos](#)

8 | Conclusiones

8.1. Objetivos Alcanzados

Una vez acabado el Trabajo Fin de Grado, considero que se han cumplido todos los objetivos planteados inicialmente para este sistema:

- **Controlar el acceso mediante reconocimiento por visión artificial.** He usado un sistema ALPR (Automatic License Plate Recognition) que utiliza modelos YOLO para la detección de vehículos y OCR para el reconocimiento de texto, el sistema procesa automáticamente las imágenes de las cámaras de entrada, salida y zonas internas.
- **Registrar de forma continua los datos para su posterior análisis.** Todas las entradas, salidas y características del vehículo detectado se almacenan automáticamente en la base de datos. Esta recopilación de información permite un posterior análisis que facilita la mejora continua del sistema y la extracción de patrones de uso.
- **Visualización del estado del parking en tiempo real.** Para facilitar la gestión, he diseñado un frontend intuitivo que muestra tanto las plazas libres por zona, como toda la información del parking en diferentes vistas. Las vistas sobre plazas libres se actualizan cada 5 segundos logrando que sea en tiempo real.
- **Accesibilidad multiplataforma.** Gracias a Flet, el sistema funciona en dispositivos móviles, tablets, ordenadores de escritorio y navegadores web y sistemas operativos, tanto Windows, Linux, IOS y Android. Lo que consigue acceder al sistema desde cualquier dispositivo o sistema.
- **Desarrollar una solución de código abierto y bajo coste.** El sistema ha sido diseñado empleando herramientas gratuitas, librerías de código abierto y hardware accesible, con el objetivo de ofrecer una alternativa funcional y económica frente a las soluciones comerciales del mercado.

8.2. Vías Futuras

De cara a una futura evolución del proyecto, las partes del sistema que se tienen que mejorar son:

- **Vinculación con el sistema de autenticación institucional de la UGR.** La evolución mas importante para este proyecto sería que, el sistema de autenticación se integrase con el utilizado por la Universidad de Granada.

Esto permitiría que el personal autorizado acceda mediante sus credenciales institucionales, evitando así la necesidad de crear nuevas cuentas y recordar mas contraseñas. Esta vinculación no solo simplificaría la gestión de permisos, sino que también reforzaría la seguridad del sistema al aprovechar la infraestructura de autenticación ya existente.

- **Mejoras en la precisión del reconocimiento de matrículas.** Aunque el sistema actual funciona con modelos preentrenados, se podría implementar un entrenamiento específico con datos de este parking para mejorar la precisión.
- **Asignación de matrículas a usuarios identificados.** Otra mejora relevante sería permitir la asociación de matrículas a personas concretas dentro del sistema. Esto permitiría generar informes por usuario y facilitar la identificación de vehículos autorizados, además se podría detectar accesos no permitidos.

En caso de que se quiera orientar el proyecto hacia un modelo de negocio de parking convencionales, se implementarían estas otras mejoras:

- **Mejoras en la precisión del reconocimiento de matrículas.** Si se utiliza este sistema en un parking comercial, se tendría que mejorar el sistema de detección de matriculas y de gestión de errores para que no cueste dinero algún error del sistema.
- **Integración con sistemas de pago automatizado.** Otra mejora sería implementar facturación automática basada en el tiempo de permanencia del vehículo calculado entre `date_in` y `date_out`. Esto permitiría generar cobros automáticos y recibos digitales.
- **Sistema de reservas y notificaciones.** Desarrollar funcionalidades que permitan a los usuarios reservar plazas con antelación y recibir notificaciones cuando su plaza esté disponible o cuando deban mover su vehículo.

Según las aplicaciones que he estado mirando para este proyecto, todas las soluciones que he encontrado se centran en aspectos comerciales como precios, localización de aparcamientos o gestión de pagos. A diferencia de estas, este proyecto se orienta principalmente a la gestión interna del aparcamiento y al aprovechamiento de los datos generados por el propio sistema.

Cabe destacar que, para convertir esta solución en un sistema comercial, gran parte de la infraestructura ya está implementada. Bastaría con desarrollar una interfaz orientada al usuario final o adaptar la existente con un enfoque más comercial. Con estas modificaciones y las mejoras anteriores, este sistema podría estar a la altura de las soluciones actuales del mercado.

8.3. Valoración de Flet

Inicialmente, para adaptar el pensamiento de diseño web tradicional (HTML, CSS, PHP) a la estructura de componentes de Flet necesité un tiempo. Sin embargo, una vez comprendida la lógica de contenedores, filas y columnas, el desarrollo se volvió más sencillo. Con la primera interfaz que implementé, que fué la de login y register, ya demostró la capacidad de este framework.

Uno de los elementos que más ha facilitado el proceso ha sido la documentación oficial del framework. Esta no solo es extensa, sino que incluye numerosos ejemplos funcionales acompañados de su código correspondiente. Gracias a esta base, a numerosos videos de YouTube y herramientas de Inteligencia Artificial, pude aprender sobre esta tecnología y construir la interfaz según las necesidades del sistema.

En resumen, me siento satisfecho con la elección de Flet para este proyecto. Haber desarrollado una aplicación funcional y completa con este framework ha sido una oportunidad excelente para poner a prueba mis habilidades que he adquirido durante la carrera.

8.4. Valoración Personal

Este proyecto ha sido una montaña rusa de aprendizaje y crecimiento. Aunque tuve que hacer algunas pausas para centrarme en mi último cuatrimestre de carrera, la idea y el desarrollo han estado conmigo desde que lo empecé.

No ha sido fácil compaginarlo con las clases, exámenes y por último con el inicio de las prácticas de empresa, pero al final he conseguido sacarlo adelante y cumplir los objetivos. Ha sido un reto grande, sobre todo porque he tenido que hacerlo por mi cuenta, sin nadie experto en Flet a quien preguntar directamente. Gracias sobre todo a la cantidad de tutoriales de YouTube, de ejemplos en GitHub e incluso a la inteligencia artificial, todo eso ha sido clave para seguir avanzando cuando me atascaba.

Lo que más valoro es lo que he aprendido por el camino. En la carrera estamos acostumbrados a seguir prácticas guiadas y contar con el apoyo de profesores y compañeros, pero aquí he tenido que organizarme yo solo y buscarme la vida. Eso me ha ayudado a ganar seguridad, a confiar más en mis capacidades y a darme cuenta de que puedo sacar adelante un proyecto grande por mi cuenta. Y la verdad, me siento bastante orgulloso del resultado.

Este proyecto me ha preparado para enfrentar desafíos más complejos y me ha dado confianza en mi capacidad para aprender y aplicar nuevas tecnologías.

Bibliografía

- [1] Amazon Web Services. ¿qué es python? <https://aws.amazon.com/es/what-is/python/>.
- [2] BairesDev. ¿qué es flask? <https://www.bairesdev.com/blog/what-is-flask/>.
- [3] Flet. Flet documentación. <https://flet.dev/docs/>.
- [4] Gobierno de España. Ley 40/2002, de 14 de noviembre, reguladora del régimen jurídico del coche oficial. <https://www.boe.es/eli/es/l/2002/11/14/40/dof/spa/pdf>, 2002. Boletín Oficial del Estado, núm. 274, de 15 de noviembre de 2002.
- [5] MongoDB Inc. Mongodb atlas pricing. <https://www.mongodb.com/pricing>.
- [6] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolov8. <https://github.com/ultralytics/ultralytics>, 2023.
- [7] Carmen Josefa Martínez Cruz. Tema 2 - sistemas de información basados en web, 2025. Material docente de la asignatura "Sistemas de Información Basados en Web", Universidad de Granada.
- [8] MongoDB. Why use mongodb? <https://www.mongodb.com/resources/products/fundamentals/why-use-mongodb>.
- [9] OpenWebinars. ¿qué es vscode y qué ventajas ofrece? <https://openwebinars.net/blog/que-es-visual-studio-code-y-que-ventajas-ofrece/#qu%C3%A9-es-visual-studio-code>.
- [10] Quercus Technologies. Aplicaciones. <https://quercus-technologies.com/es/aplicaciones>, 2025.
- [11] Rekor Systems, Inc. Rekor scout®. <https://www.rekor.ai/software/scout>.
- [12] Jaime Requeijo. Lo bonito de la tecnología es hacer la experiencia fácil para que sea mágica. https://www.elespanol.com/omicron/software/20221017/jaime-requeijo-easypark-bonito-tecnologia-experiencia-magica/709679168_0.html, 2022.
- [13] Telpark. Entrada express - paga el parking con el móvil. <https://www.telpark.com/es/productos-parking/entrada-express/>, 2025.

- [14] Domingo Torrens. Mongoddb atlas: Maneja mongoddb en la nube. <https://iconotc.com/mongoddb-atlas-nube-mongoddb/>.
- [15] Ultralytics. ¿qué hace ultralytics? <https://www.promptloop.com/directory/what-does-ultralytics-do>.